MAE 162E Final Report:
Stacked Disk Transport Rover

Instructor: Yen-Chi Chang
TAs: Kevin Lai, Ting-Hao Liu

Group 10
Justin Lee: 105587052
Matthew Tamin: 006042107
Matthew Hutchinson: 505621252
Jacob Sayono: 505368811
John Lee: 605532726

MAE 162E: Mechanical Product Design - II
Spring 2024

Mechanical & Aerospace Engineering Department
University of California, Los Angeles

June 14, 2024

The Team

Group 10 (pictured from left to right): John Lee, Jacob Sayono, Justin Lee, Matthew Tamin, Matthew Hutchinson

Abstract

The purpose of this report is to present a concept design for a rover robot that will complete the required tasks of going through the venue to transport the disk-shaped objects from the start to the finish point. To ensure that the rover can handle the given tasks, we have designed a cart mechanism that will allow the disk-shaped object to slide with the help of our scooping arm mechanism, and to be delivered using a rising platform and pushing arm design. To ensure that the fabrication of this design is possible, the project requires a deeper level of understanding of IoT machines, utilizing microcontrollers such as Arduinos and minicomputers like a Raspberry Pi. These controllers are the brains of the rover and are responsible for commanding the motors and sensors to perform the task smoothly.

Table of Contents

The Team ii

Abstract iii

List of Figures vi

List of Tables vii

List of Symbols viii

1. Introduction 1
   - 1.1. Literature Review 2
   - 1.2. High Level-Design Requirements 4
   - 1.3. Low-Level Design Requirements 4
2. Design Description 5
   - 2.1. Concept 1 5
   - 2.2. Concept 2 6
   - 2.3. Concept 3 6
   - 2.4. Pairwise Comparison Chart and Objectives Tree 7
   - 2.5. Design Concept Selection 8
   - 2.6. Initial Design Choice Description 9
   - 2.7. Finalized Project Design Description 9
   - 2.8. System Specifications 10
   - 2.9. Mechanical Systems 11
   - 2.10. Control Systems 12
3. Subsystem Design Description 12
   - 3.1. Structural Subsystem 12
   - 3.2. Mechanical Subsystem 13
   - 3.3. Electronics Subsystem 14
4. Design Analysis 14
   - 4.1. Preliminary Calculations 15
     - 4.1.1. Drive System Power Requirements 15
     - 4.1.2. Move Profile & Motor Torque Requirements 16
   - 4.2. Basket Lift Calculations 19
   - 4.3. Scooper Arm Calculations 19
   - 4.4. Bottom Plate FEA Study 20
5. Control System Design 21
   - 5.1. System Architecture 21

5.2. Closed-Loop Control of Differential Drive Motors with Encoders 22
5.3. Wheel Velocity Calculations and Commands 24
5.4. Camera System 25
5.5. Arm Mechanism 30
5.6. State Diagram 33
6. Product Fabrication 34
6.1. Chassis Fabrication 34
6.2. Basket Lift Fabrication 35
6.3. Scooper Arm Fabrication 35
6.4. Final Product 35
7. Product Performance Testing and Evaluation 36
7.1. Runtimes 37
7.2. Overall Performance 37
8. Work Breakdown Schedule 37
8.1. Winter Quarter 37
8.2. Spring Quarter 37
9. Bill of Materials and Cost Analysis 38
10. Design Requirement Satisfaction 39
10.1. High-Level Design Requirements 39
10.2. Low-Level Design Requirements 39
11. Summary and Conclusions 40
12. References 41
13. Appendix 42
13.1. Initial Preliminary Calculations 42
13.1.1. Device Weight Estimate 42
13.1.2. Drive System and Power Calculations 43
13.1.3. Move Profile and Motor Sizing Calculations 44

List of Figures

Figure 1: Model of the testing track for the designed rover 1
Figure 2: Adeept Smart Robot Kit and Lynxmotion A4WD1 Rover Kit assembly 2
Figure 3: Elegoo Smart Robot Kit and DIY Arduino Line Follower Robot 3
Figure 4: CAD Render of Robot Arm and Gripper System with a Rack and Pinion Mechanism. 3
Figure 5: Design Concept 1 - An illustration of a robot with a simple arm lift and claw mechanism. 5
Figure 6: Design Concept 2 - A sketch displaying a robot with a suction cup attached to an arm lift that has two degrees of freedom and cart.. 6
Figure 7: Design Concept 3 – CAD model of a robot with a scooper arm, a storage cage, and an iris mechanism. 7
Figure 8: Iris mechanism incorporated with Design Concept 3 7
Figure 9: Initial Robot Design CAD Model with Location for CoG 9
Figure 10: Current Robot Design CAD Model 10
Figure 11: Side View of the Crank-Shaft Mechanism with Improved Visibility 11
Figure 12: Arm Mechanism at Default and Fully Extended Positions 12
Figure 13: Electronics Schematic 14
Figure 14: Required Minimum Coefficient of Friction. 15
Figure 15: Track Divided into Segments 17
Figure 16: Move Profile. 18
Figure 17: Bending Displacement Study for Bottom Plate. 20
Figure 18: Factor of Safety Study for Bottom Plate. 21
Figure 19: Communication Network System. 21
Figure 20: Incorporating Computer Vision with Motor Steering Control. 25
Figure 22: State Diagram of Rover Robot Design. 33
Figure 22: Overhead View of Fully Assembled Rover Design. 36
Figure 23: Bill of Materials for Final Rover Design. 38

List of Tables

Table 1: High-Level Design Requirements 4
Table 2: Low-Level Design Requirements 4
Table 3: Pairwise Comparison Chart 8
Table 4: Objectives Tree 8
Table 5: Component Weights and Total Device Weight 15
Table 6: The estimated amount of time for acceleration, constant velocity, and deceleration of each segment and each segment's move profile type 17
Table 7: The estimated maximum velocity and acceleration for each segment 17
Table 8: The estimated maximum required propulsion force for each segment (summation of inertia force, weight force, wheel friction force, and rolling force) 18
Table 9: The estimated required propulsion power for each segment 18
Table 10: Estimated maximum required propulsion power and motor torque along each segment 19

List of Symbols

Symbol	Definition
F_i (N)	Inertia force
F_w (N)	Weight force
F_f (N)	Wheel friction force
F_{rol} (N)	Rolling force
F_{prop} (N)	Propulsion force
L_{sg} (m)	Total segment length
$\eta_{drive-system}$	Drive system efficiency
P_{prop} (W)	Propulsion Power

Symbol	Definition
t_a (s)	Acceleration time
t_c (s)	Constant velocity time
t_d (s)	Deceleration time
t_{tot} (s)	Total time
T_{prop} (N-m)	Motor torque
μ_f	Wheel friction coefficient
μ_{rol}	Rolling friction coefficient
d_{tot} (m)	Total distance

1. Introduction

The main problem to be solved within this project is to be able to stack and transport multiple disk-shaped objects in an economical, safe, cost-friendly, and aesthetically acceptable manner. To be more specific, the product must be able to navigate through an obstacle course to multiple pre-determined locations, where it must pick up and stack at least two disks of 3.5-inch diameter. To add to these requirements, the product must then transport these disks safely through a track designed with an inclining and declining ramp of fifteen degrees, as well as a curved path. Upon completing the course, the product must then be able to drop off the disks in a pre-designated order, and then optionally navigate back to the start of the looping track course.

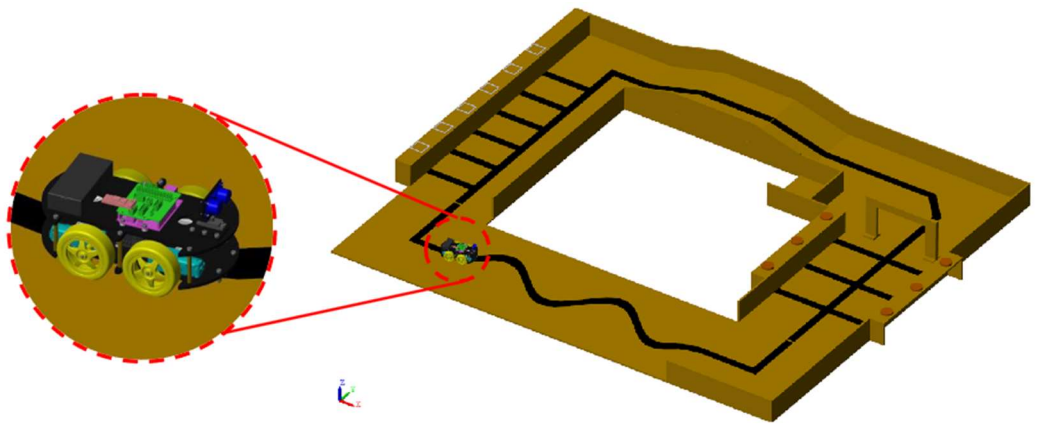

Figure 1: Model of the testing track for the designed rover

Besides the deliverables that the product must be able to make, there are also a few physical restrictions and guidelines that the product must meet. To start, the entire product must be able to be contained within a 30x25x20 cm volume, as defined by the length, width, and height in that order. The product must also be able to withstand a simple drop test, at which it is to be dropped at a height of roughly six inches and remain functional after the test with no major broken parts.

Going back on the basic functions of the product, it must be able to pick up and stack at least two disks from an elevated platform and transport them through a course consisting of a ramp and curved path, after which the product must safely drop off the disks onto an elevated platform in a pre-determined order. A product designed with these specifications should be able to deliver a solution geared towards transporting multiple disk-shaped objects from scattered locations to a single set location within a single run. Instead of a solution consisting of multiple stops at the final location, this solution is capable of transporting the combined total deliverable to the intended destination in a condensed run. For example, putting this into a real-world example, a similar product would be able to pick up various components of a hamburger from various stations within a kitchen before presenting it to the customer as a complete meal.

Thinking about this project as a marketable product, there are a couple of additional requirements that a client would realistically expect the product to meet. To start, the product itself needs to be visually presentable and aesthetically pleasing, such that a consumer not familiar with the product would be able to identify its general purpose and be able to trust that the product will run as intended. In addition, the product should be designed so that it can safely be operated. There should be no exposed hardware that may damage the consumer or user, the electronic components that may cause any electrical damage should be properly and safely contained, and there should be an option to shut

off the product (i.e. remove the battery) in the case that an emergency does occur. Also, none of the materials used should be toxic, easily flammable, nor should it be an irritant. These requirements should be met so that the product fits within the basic standards set by products already on the market.

Similar products do exist on the current market, as they meet the goal of being able to pick up and transport objects. They are often marketed as “Rover Kits” or “Robot Arm Kits with Wheels” online and are capable of picking up items with a robot manipulator arm that is attached to a rover. As a result, these products are able to transport objects from one location to another. The basic design of an arm capable of picking up objects attached to a rover capable of moving the arm assembly is similar to the overall design that is presented for this project. Two examples of currently marketed products that achieve similar solutions as the one presented within this project are shown in the figure below.

Figure 2: Adeept Smart Robot Kit (left) and Lynxmotion A4WD1 Rover Kit assembly (right)

Despite being similar in overall design and functionality as the product designed as part of this project, there are a number of key differences. In general, the product needed for this project must have more adaptability and capability than the products listed above. The other products that were found to already exist on the market are normally capable of transporting one object at a time within the given specifications. So, in order to meet the requirements of stacking the disks, the existing products would need to either perform multiple cycles throughout the track or risk having the objects fall, as the grippers are more suited for singular, simpler objects. This setback would waste time and the overall design could be further optimized for the specialized task at hand. More capable designs that are already on the market tend to face the problem that they are too big for the given volume restrictions or are too costly. For context, the finished product should not cost more than five hundred dollars for the entire project. As such, our product solution is one that not only meets the specific design requirements, but it is also designed to be a more economical solution for the given task.

The current approach for our project solution is to design a custom rover and robot arm mechanism from scratch using components from various vendors and manufacturers. The overall design gains inspiration from currently existing products on the market, but with modifications to allow for the product to perform specific actions as outlined by the project specifications. In order to provide a sense of guidance for the product design roadmap, an acceptable design is currently established as one that can safely transport two stacked disks from the start to finish location within a single five-minute run.

1.1. Literature Review

Upon looking through the market, there are plenty of similar products that achieve similar goals. To break these products into two general parts, there is the line following rover and the robot arm

manipulator. Focusing on the former, there are plenty of resources online for kits and do-it-yourself projects for a simple line following rover power by an Arduino. Examples of such kits are shown in the figure below.

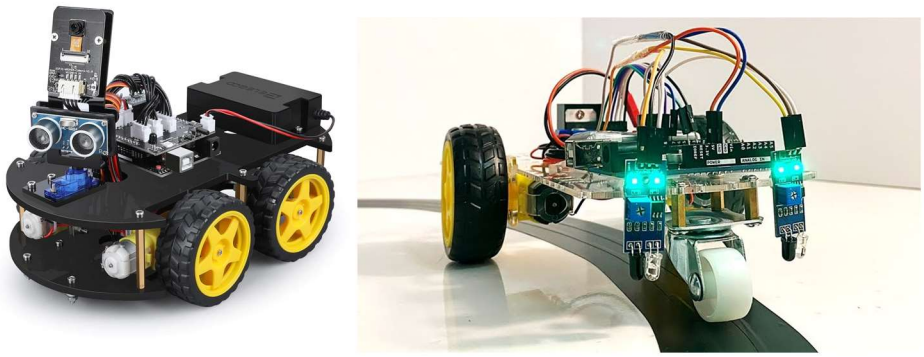

Figure 3: Elegoo Smart Robot Kit (left) and DIY Arduino Line Follower Robot (right)

The kit pictured on the left for Figure 3 shows the Elegoo Kit that was provided during the weekly lab sessions. With limited exposure to the components and capabilities of these kits, it was found that an IR sensor is sufficient to allow the rover to navigate a course with simple dark lines. However, one definite limitation to the kits were that they struggled to perform sharp turns and track a sharp corner as a turn. Based off this drawback, our group is considering incorporating multiple IR sensors. The thought process behind this design change is that if the nodes are spaced for a wide enough range, then the rover should be able to differentiate between soft turns and sharp turns, making it more accurate than the provided kit. The implementation of this design is also being considered with the idea that it may help to identify crossroads, such as those found near the pickup and drop off platforms.

Moving onto the second half of the robot, the arm was designed around translation degrees of freedom. This was done to simplify the maneuver for picking up and dropping off the disks to be simple pushing and pulling motions. In general, a rack and pinion mechanism is the current option being considered for these motions. An example of a robot arm design that incorporates this mechanism is shown below.

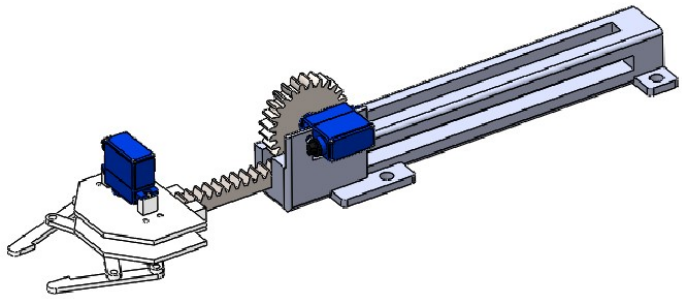

Figure 4: CAD Render of Robot Arm and Gripper System with a Rack and Pinion Mechanism

To generalize the thought behind the design process, rotational degrees of freedom were avoided to make the robot arm as simple as possible to develop and maintain. It was determined that a robot arm with rotational degrees of freedom, namely at its base and elbow, would pose an issue in terms of durability. The main concern was the fact that a motor would be located along the middle of the robot arm, and the team determined that having any issues occur with the motor during a drop test would prove fatal for the overall function of the robot.

1.2. High Level-Design Requirements

The overall design restrictions set by the consumer are called High-level Design Requirements, and they largely determine how a product is to be designed. For the current design project, the High-level Design Requirements are as follows, in order from highest priority to lowest priority:

Table 1: High-Level Design Requirements

HLDR*	Description	Priority
1	Fit within volume of 30x25x20 cm (LxWxH)	High
2	Rover can function after being dropped from a height of 6”	High
3	Rover navigate through black strip (2 inches) from starting location	High
4	Rover can pick up, assemble in a specified order, and drop off disks onto an elevated platform	Medium
5	Avoid dynamic obstacle autonomously	Medium
6	Rover carries weights onto a 15-degree ramp with weights	Medium
7	Navigate through a curved path without getting lost	Medium
8	Product looks marketable and aesthetically pleasing	Low

These High-level Design Requirements are the restrictions and design specifications provided within the project guidelines. These requirements are not determined by the design team, but instead by the customer or party requesting the design.

1.3. Low-Level Design Requirements

Aside from the design requirements set by an outside party, the design team also sets restrictions for themselves – called Low-level Design Requirements, to allow for improvements and clarifications to the High-level Design Requirements. The Low-level Design Requirements are as follows, in order from highest priority to lowest priority:

Table 2: Low-Level Design Requirements

LLDR*	Description	Priority
1	Fewest number of DOFs and motors	High
2	Lowest possible center of mass	High
3	Length and width close to dimension restrictions (for more stability while moving and turning)	Medium
4	Fit into volume with 5 mm tolerances on all sides	Medium
5	Perform the task within a span of five minutes	Medium
6	Modular design for ease in exchanging and repairing parts	Low

These Low-level Design Requirements were generally created by the design team for the project, with a focus on simplicity and low-risk designs. The high priority items were decided to restrict the design to be as simple to design and easy to run. In general, less DOFs and motors allows for easier development, while a lower center of mass allows for more stable movement. The medium priority tasks were centered around ensuring that the product runs well within the given specifications. Finally, the low priority tasks were created for ease-of-life, especially considering that the next stage following the conceptual design will be product development and testing.

2. Design Description

This section centers around the conception of the current robot design, starting with three separate design concepts for the robot. Each design concept serves to provide a unique approach to the task of moving the desks. Based off various factors and weights of each factor towards the end design, one of the three concepts were chosen and built on. The preliminary design for the robot is described and a visual model is presented at the end of the section. Then, the finalized design is given and any changes present within it are explained.

2.1. Concept 1

Concept 1 features a straightforward design consisting of a basic arm lift equipped with a claw. During our first meeting, it was decided that reducing the amount of unnecessary movement is important. This concept emphasizes the minimalistic motion discussed in our meeting, utilizing only vertical translation of the arm to engage with the disks. By limiting the robot's degrees of freedom, the design simplifies the construction and function, resulting in a cost-effective and efficient solution. The arm descends directly above the disk and then employs the claw to grasp and lift the object with ease. Overall, the design concept is a simple systematic solution to the project's task and requirements.

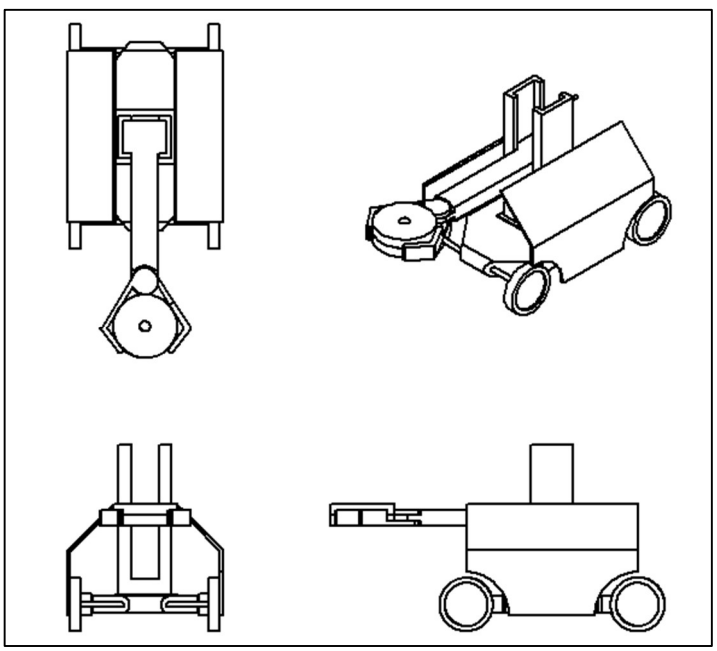

Figure 5: Design Concept 1 - An illustration of a robot with a simple arm lift and claw mechanism

Going over the design, the strength of concept 1 is that it is simple and easy to design and develop. However, its weaknesses also lie in its simplicity. As it has simple translational DOFs, the rover must be positioned with exact precision in order to pick up the disks, to stack separate disks, and to pick up this stack. Also, the height of the required tower is likely to raise the center of mass significantly, which is against one of our Low-level Design Requirements. The claw itself also lacks support for the disks, which may slip and fall while in transport. Overall, the design is a solid and simple basis to build from, but with very little room for error, modifications and improvements may be necessary to meet all the design requirements.

2.2. Concept 2

The idea of stacking disks and mobilizing them in one manner was discussed in our following team discussions. This second design concept introduces an arm lift with two degrees of freedom, paired with a cart attached to the chassis. At the end of the arm lift is a suction cup that is attached with a free joint. This design allows the robot to carry multiple disks by stacking them within the cart. Consequently, the robot can transport several disks simultaneously to the desired location, significantly reducing the number of trips required across the track and enhancing overall efficiency. Additionally, the integration of a free-hanging suction cup with the cart allows greater tolerance for inaccuracies and simplifies the programming required for the robot's autonomous operations. The final delivery of the disks is the reverse of picking them up, in that the suction cup is used to move the disks out of the cart and onto the platform.

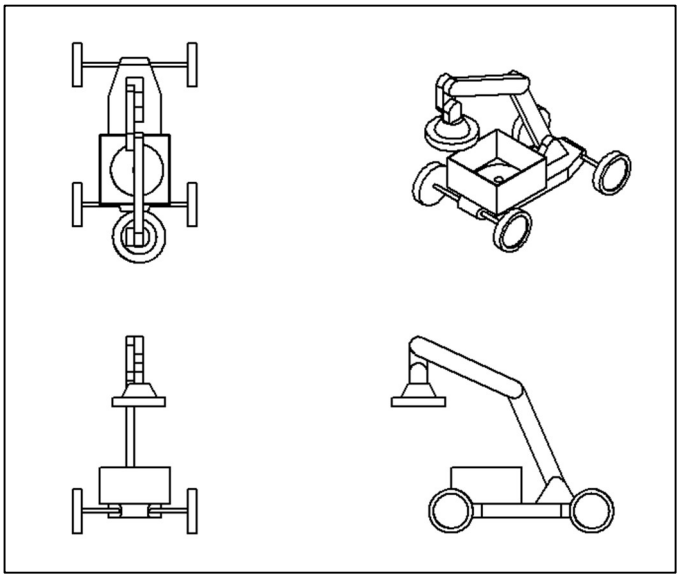

Figure 6: Design Concept 2 - A sketch displaying a robot with a suction cup attached to an arm lift that has two degrees of freedom and cart

Focusing on the pros of concept 2, compared to concept 1, the ideal suction cup allows for the disk to be carried with little worry as to whether it may fall when being picked up. Also, the cart ensures that the disks will not fall when traveling around the course. However, moving onto the cons of this concept, the lack of a wide range of motion makes this concept very weak in terms of adaptability. Also, the need to use the suction cup to deposit the disks also requires the disks to be picked up in reverse order, which may be a hassle. Approaching the design from another perspective, the extended arm is also very thin in terms of its length, which calls into question the strength and durability of the arm. In the case of a drop test, the arm itself is susceptible to bending and breaking. Overall, the design itself is a capable one, but the addition of protective shielding may be beneficial.

2.3. Concept 3

Concept 3 features a design consisting of a cart in the front of the rover and a sweeper claw. Along with that is an iris mechanism that will allow for a safer drop-off of the disks onto the platform. This cart mechanism is very effective, especially for handling multiple objects, like the cart in concept 2. While the objects are going to be disk-shaped, we decided to design a square shaped cart to ensure that the object will not be likely to get caught on the sides of the cage, and for simplicity. As there is one claw and an iris mechanism, there will be minimal need for motors, as the biggest concern would

be to pick the objects up, as it is theoretically simple to drop off the disks.

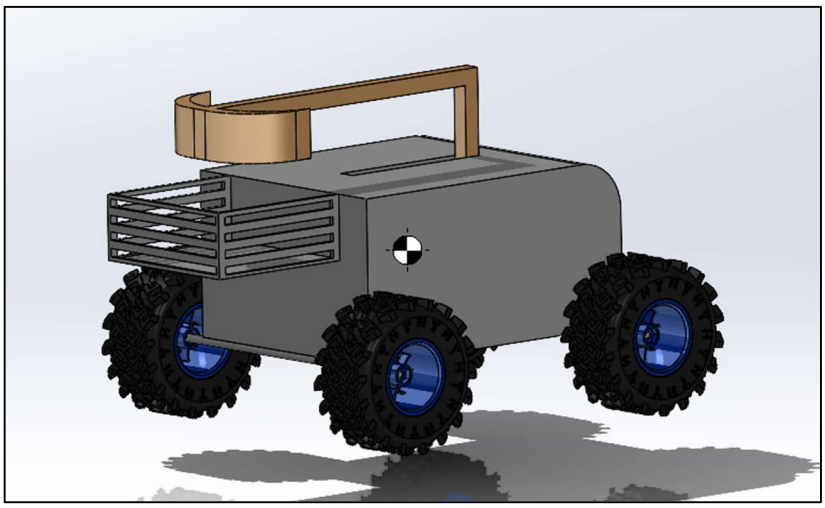

Figure 7: Design Concept 3 – CAD model of a robot with a scooper arm, a storage cage, and an iris mechanism

The strengths of concept 3 are that there are not very many moving parts and DOFs, which makes the design simple to develop. Also, the use of an iris mechanism, as shown in Figure 6 below, allows for a low-risk and easy to design method of dropping off the disks. However, that does not mean it does not have its weaknesses. This design would need to raise the cart significantly in order to drop off the disks using the iris mechanism, which would then raise the center of mass. Especially with the load situated towards the front of the rover, this may cause tipping if not properly calculated. Yet, overall, this design is the most adaptable of the three presented thus far, but also requires the most amount of foresight to become a reliable design.

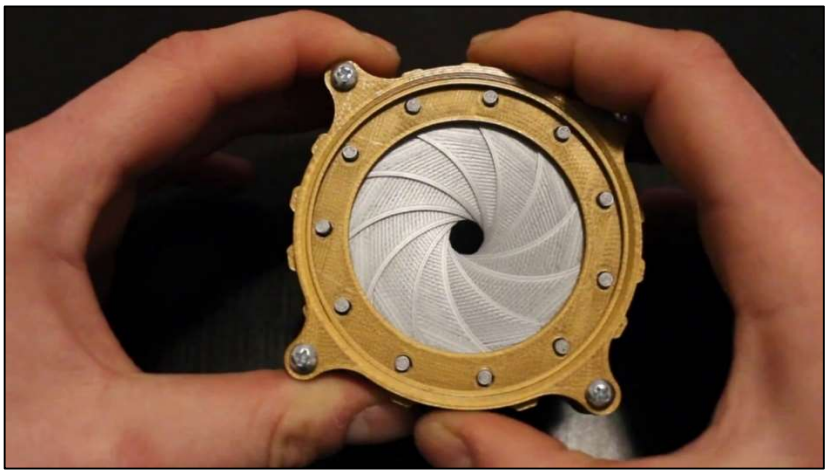

Figure 8: Iris mechanism incorporated with Design Concept 3 [4]

2.4. Pairwise Comparison Chart and Objectives Tree

In order to determine which concept is best suited for the specific project requirements, the team first created a pairwise comparison chart, as shown in Table 3 below. A total of seven factors were selected and voted between in order to determine which individual factors held more or less weight

in determining which concept design to proceed with.

Table 3: Pairwise Comparison Chart

Pairwise Comparison Chart											
# of Designers: 5			Compared To:								
	Is this factor used? (YES = 1, NO = 0)	Factors	A	B	C	D	E	F	G	Total	Weight
Asset	1	A. Cost	5	4	2	1	4	2	3	21	0.15
	1	B. Weight	1	5	2	1	3	2	3	17	0.12
	1	C. Safety	3	3	5	1	4	3	3	22	0.16
	1	D. Durability	4	4	4	5	4	3	3	27	0.19
	1	E. Aesthetics	1	2	1	1	5	1	1	12	0.09
	1	F. Maneuverability	3	3	2	2	4	5	2	21	0.15
	1	G. Manufacturability	2	2	2	2	4	3	5	20	0.14
Team 10, MAE 162D					Total Sum:					140	1.00

After finding these weights, the values from the pairwise comparison chart were transferred to an objectives tree. The three design concepts were rated using the aforementioned factors, which were then weighted and totaled to find each design's total "design satisfaction" score. The table of calculations is shown in the table below.

Table 4: Objectives Tree

Objectives Tree							
		Raw Design Scores			Weighted Design Scores		
Design Objectives Factor	PCC Weight	Design 1	Design 2	Design 3	Design 1	Design 2	Design 3
Cost	0.15	7	4	7	1.0500	0.6000	1.0500
Weight	0.12	7	6	7	0.8500	0.7286	0.8500
Safety	0.16	4	2	7	0.6286	0.3143	1.1000
Durability	0.19	6	2	9	1.1571	0.3857	1.7357
Aesthetics	0.09	2	8	7	0.1714	0.6857	0.6000
Maneuverability	0.15	6	6	6	0.9000	0.9000	0.9000
Manufacturability	0.14	8	6	7	1.1429	0.8571	1.0000
Total Weight:	1.00		Total Score:		5.9000	4.4714	7.2357

Using the scores listed in Table 4, the team decided on a design concept to proceed with for the time being, as will be explored in the next section. It should be noted that, as the design shifts from concept to development, more factors may be added to account for design aspects that the team had not previously noticed. As such, the design concepts may vary from their initial conceptions, and this information will be provided in later reports.

2.5. Design Concept Selection

Between the three concept designs provided in Section 4.1 to 4.3 above, the team decided to proceed with Concept Design 3. As provided in Table 4, design three resulted in the highest total "design satisfaction" score among the three designs. The other two designs had scores of approximately 5.9 and 4.5, while design three had a score of 7.2. The design itself is not perfect, as the

team hopes to improve upon it for a score of at least 8.0, but it was deemed enough to proceed with the design process for now.

As it currently stands, design three is the best among the three because it has similar strengths to the other two without as many weaknesses. Its design retains the simplicity of the other two, the use of less motors as in the first design, and the stability in transporting the disks that is found in the second design. While maintaining these strengths, the third design is more durable than the second design, and it also is more adaptable than the other two designs. With these factors in mind, the design team determined that the third design would be the most effective in performing the tasks specified by the project specifications.

2.6. Initial Design Choice Description

The initially chosen design centers its main components from the third concept design. For much of the robot, the design was kept the same regarding the approach and the implementation of the robot. However, the iris mechanism mentioned in Section 4.3 was abandoned due to the fact that keeping it in the design would mean that an additional motor would be necessary for it to function. Rather, the concept design was altered so that the bottom surface of the cage would translate up and down, while the sides of the cage would remain stationary. A CAD model of this version of the robot design is shown below.

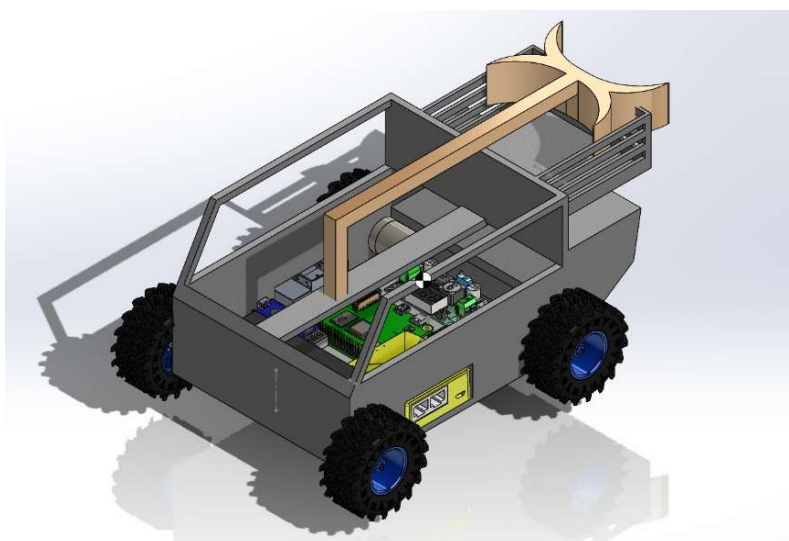

Figure 9: Initial Robot Design CAD Model

The idea behind this change is that the platform will be able to raise the disks to the same height as the elevated drop-off platform. As such, the same arm mechanism that was used to push the disks into the cage will be able to reverse its motion and push the stacked disks onto the elevated platform. There was an initial discussion about the risk of the object tipping, but this concern was quickly dismissed in light of the fact that the height of three stacked disks is still less than half the diameter of a disk. The disks were then determined to be plenty wide enough to not tip over while being pushed, even in the case of three stacked disks.

2.7. Finalized Project Design Description

The initial design was altered due to issues involving its fabrication and a lack of understanding on how the mechanisms planned for the pickup and drop off processes were meant to be incorporated

into it. The finalized design was created using the existing drive system and the various components within it, including the motors and the electronics, as its central reference. The basic idea behind the chassis was also changed to use the Elegoo kit provided during class as the main reference. A figure depicting the CAD model of the final design is shown below.

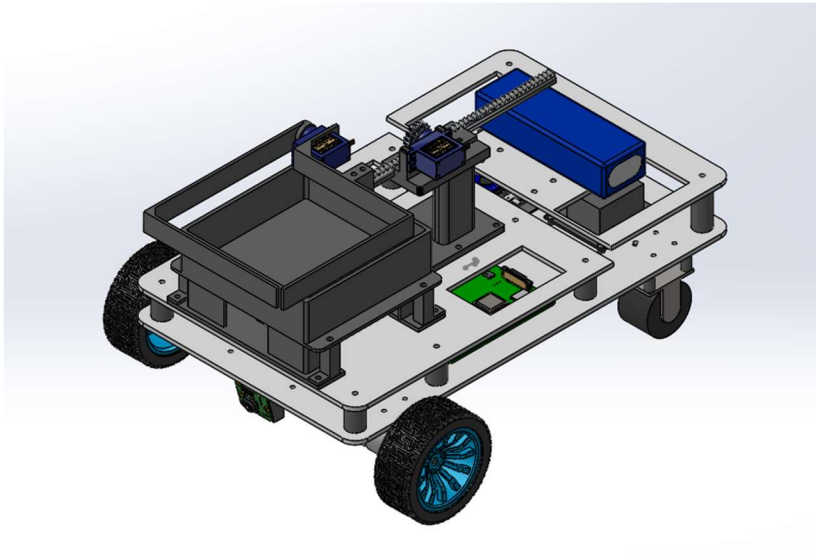

Figure 10: Current Robot Design CAD Model

As can be seen from the figure above, the design maintains the basic basket-lift and scooper-arm logic behind its mechanical design. The basket-lift was created such that a crank and slider mechanism could raise the platform between two heights, of which one would be the appropriate height for picking up the discs and the other would be the right height for sliding the disks off of the platform. The scooper arm was changed from a rigid arm with two translational degrees of freedom to a scooper with one translational and one rotational degree of freedom. The main inspiration for this change was to reduce the amount of mass that each motor was designed to displace, as the initial design required a motor to translate the entire arm design. Finally, the chassis itself was changed from a hollowed-out box design to a design centered around layers held apart by standoffs. This design change was made to achieve an easier fabrication process for the rover, as well as to allow for easier changes in the device's height in case our team's understanding of the competition venue's dimensions was incorrect.

2.8. System Specifications

The rover has dimensions of 29.972 x 24.143 x 16.417 cm (LxWxH). The chassis is constructed from 3 mm thick acrylic sheets, measuring 17.78 cm in width and 29.972 cm in length. It features a wheelbase of 24.143 cm with a front-wheel drive system powered by 12V motors operating at 130 rpm, paired with 65 mm wheels. For enhanced maneuvering, the rear wheels include 1-inch swivel caster wheels.

The basket-lift mechanism is driven by a 5V positional servo motor (SG90) that rotates the crank, allowing the basket to move between a lowest position of 10.923 cm and a highest position of 13.521 cm relative to the ground. The arm system utilizes two servo motors: a 5V positional servo motor (SG90) for vertical motion of the scooper arm and a 5V continuous servo motor (FS90R) for a translational motion of 14.2 cm. The basket-lift mechanism weighs around 128.29 g and the arm system weighs around 99.74 g without screws and nuts.

Power is supplied by a 12V rechargeable battery, regulated by a buck converter to distribute 12V and 5V to the respective components. The rover is equipped with L298N DC motor controllers with encoders for precise motor control. Low-level control is managed by an Arduino Nano, while high-level computation and image processing are handled by a Raspberry Pi with an attached Raspberry Pi camera.

2.9. Mechanical Systems

The main mechanical systems within the scope of our rover design are the crank-and-shaft mechanism for the raising platform and the rack-and-pinion mechanism for the scooping arm. The raising platform relied on the crank-and-shaft mechanism to drive the platform between two vertical positions: one for picking up the discs and the other for dropping the discs off. The crank and shaft, which was connected to a positional servo motor, would rotate between two rotational positions. This correlated to two different positions for the connecting rod, which then drove the platform to alternate between two different heights that were distanced one inch apart from one another. This general mechanism assembly is shown below.

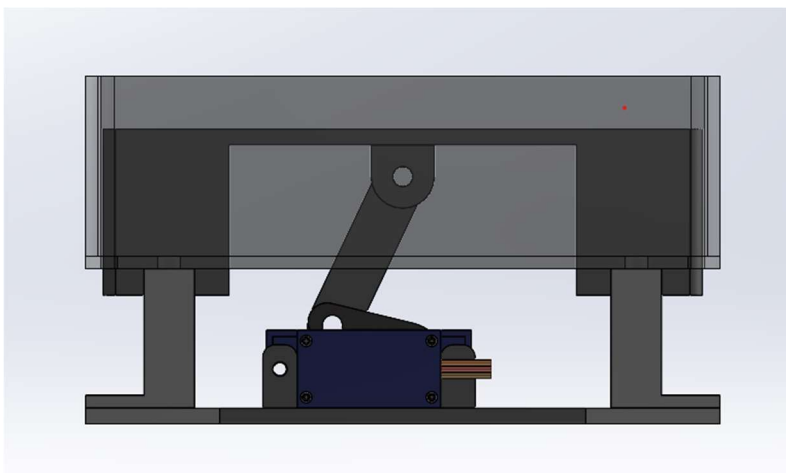

Figure 11: Side View of the Crank-Shaft Mechanism with Improved Visibility

In general, the platform that the discs sat on was an enlarged slider component for the mechanism. When the crank was set to its default position, the platform sat at the bottommost height for the mechanism design, whereas if it were rotated by 180 degrees, the platform would sit at a height of approximately one inch above its minimum height. This range of motion allowed for two disks to sit comfortably within the designated basket design, while the third and extra disk would be secured by the scooper mechanism shown below. The sides of the platform were extended to allow for alignment between the platform and the basket to ensure that the platform's main surface was relatively horizontal throughout its range of motion.

The rack-and-pinion mechanism was the central mechanism for the scooping arm design. The pinion or gear was attached to a continuous servo motor to allow for multiple revolutions, and the rotational motion of the gear would move the rack translationally. The entire design was made to provide a 150-mm stroke, which provided approximately 10-mm of leeway distance away from the pickup platform to still perform a successful pickup operation. The entire motion of the mechanism was purely in the horizontal plane, such as to provide a linear extension and retraction motion. This was intended to allow for the disks to be consistently pushed or pulled. A rotational degree of freedom was added to the arm end, at the end of the rack, to allow for the scooper to open and close. The motion of the combined mechanism is shown below.

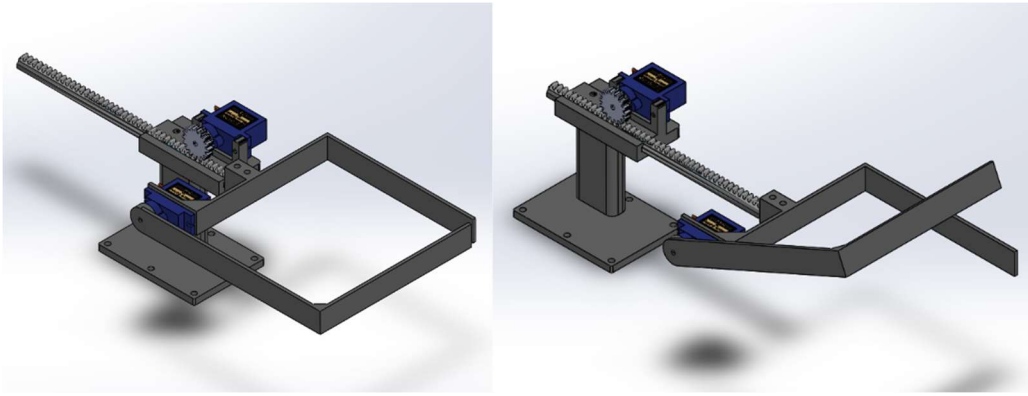

Figure 12: Arm Mechanism at its Default Position (left) and Fully Extended Position (right)

This added hinge relied on a positional servo motor that altered between zero degrees, which gave the scooper a closed perimeter, and thirty degrees, which allowed for up to three disks to be moved without knocking them over. As this hinge design was added to the end of the rack-and-pinion mechanism, one concern was the added weight would cause the rack to sag significantly. To reduce this possibility, the hinge design was made to be as thin as possible without breaking to reduce the total mass. Though there was also a concern of whether the scooper could then move the disks, the disks themselves were determined to be light enough to still be transported using this mechanism. Overall, although there were some factors that the arm mechanism skipped out on, the holistic mechanism was able to function as intended.

2.10. Control Systems

The control system for the rover design largely revolved around the use of a Raspberry Pi camera, which was meant to be responsible for line detection, object avoidance, detecting intersections for dropping off and picking up the disks, and detecting sharp turn points. The camera and the servo motors used for the disk delivery process were both connected to the Raspberry Pi, which provided the component with power and allowed for data input. So, in terms of the sensors and motors used, the single sensor used within the design was a Raspberry Pi camera, while the motors used consisted of two positional servo motors, one continuous servo motor, and two 12V motors with encoders that were meant to drive the rover.

3. Subsystem Design Description

The overall design of the rover is divided into three main parts: structural, mechanical, and electronic. The details of the design in regard to each section are split up and provided below.

3.1. Structural Subsystem

The structural subsystem of the rover was designed in a way such that the height of the robot could be easily changed and to allow for easier compartmentalization of the electronic components. All the electronic components were mounted onto a 3-mm thick acrylic plate that was laser cut to be X-mm wide and X-mm long. The thickness was chosen to allow for the sheet to flex slightly when dropped to absorb the impact, but not enough to the point where the electronic components were at severe risk of damage. Based off of the minimal amounts of damage noticed on the rover after the drop test, it can be stated that the thickness choice was suitable for this application. The chosen standoff heights

were 20-mm for the main chassis and 32-mm for the electronic storage area.

The front wheel drive (FWD) design was chosen because the calculations performed on the initial design suggested that enough torque could be produced with the 12V motors in all wheel configurations (AWD, FWD, RWD). The FWD option was also chosen due to the rover being able to turn more easily with only two wheels rather than with four. Each motor was mounted to the bottom acrylic plate using the motor mounts provided by the vendor. The motors were placed up front where the camera would be, so that the rover would turn approximately in the area that the camera would detect an intersection or sharp turn. The rear two wheels were situated with free swivel caster wheels to provide free motion for those wheels, both in turning and rolling.

At the rear of the rover, the top acrylic plate was divided and elevated using higher standoffs, as mentioned above. This was done to provide more storage for the taller-sitting electronic components, as well as to provide protection for them in terms of outside debris or in the case of impacts. The battery was mounted onto the top surface, as the increased height allowed for comfortable placement of the cables and the free area was suitable for a component that needed to be easily accessible but also well supported. There were plans to create a box using 3D printed PLA in order to make the aesthetics of the rover design better, but there were manufacturing and timing issues related to it. It would have also interfered with the mechanisms, so the box cover design was never fully pursued.

3.2. Mechanical Subsystem

The mechanical subsystem consists of a basket-lift and rack-and-pinion mechanisms. The basket-lift mechanism is first described. The basket-lift system employs a crank and slider mechanism to elevate disks for drop-off. A 5V positional servo motor (SG90) powers the crank, rotating it a full 360 degrees. The basket's outside rim stabilizes the platform that holds the disks, allowing it to move up and down with a stroke of 2.598 cm (1.022 inches), approximately the height of two disks. The rim is designed as a slim, tight fit with the platform, minimizing friction and functioning as a slider. As the crank rotates, the connecting rod pushes and pulls the platform. At 0 degrees, the platform is at its lowest height of 10.923 cm relative to the ground, and at 180 degrees, it reaches its highest point at 13.521 cm. The control system commands the servo to move 180 degrees each cycle, ensuring a straightforward and consistent operation.

The scooping arm system uses a rack-and-pinion mechanism, combined with a simple rotating powered hinge joint to collect and drop off the plastic disks. A continuous 5V servo motor (FS90R) drives the rack-and-pinion mechanism, as the gear for the mechanism is connected onto the motor. The motor completes approximately one and three quarters revolutions to cover the entire distance of the rack, until it reaches the physical stoppers located at each end of the rack. This allows for the mechanism to achieve a 150-mm (5.9 inches) stroke. However, due to the stopper design, the total translational displacement of the scooper arm is closer to 145-mm (5.7 inches). The total displacement in one direction is achieved consistently within a period of approximately eight seconds. For the hinge, a positional 5V servo motor (SG90) is mounted onto the scooper fixture, which then connects to the closing fixture. The system is designed to rotate upwards and downwards a consistent amount, which was set to 30 degrees for this project, to ensure that objects can be gathered and dropped off with the mechanism.

3.3. Electronics Subsystem

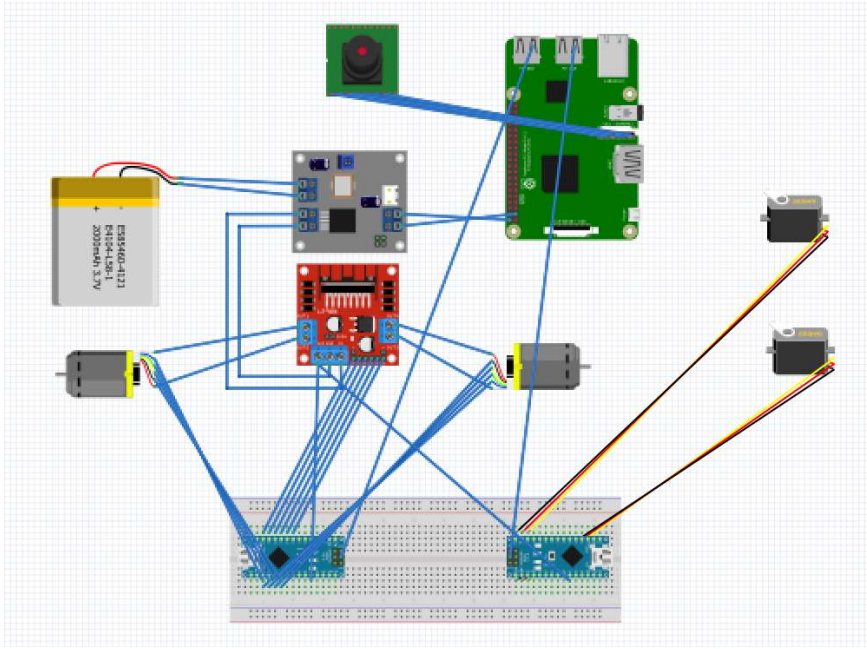

Figure 13: Electronics Schematic

The schematic of the major electrical components found within the rover design are provided in the figure above. Key notes about the diagram above is that only one Arduino Nano was used for the two 12V motors for the drive system. The second Arduino Nano did not arrive in time for the testing process of our design, so the servo motors were connected directly to the Raspberry Pi. Also, although not depicted on the diagram, there was a 10-amp fuse and switch placed between the 12V battery and the voltage regulator for safety measures. This design did end up potentially saving our electronics subsystem on at least one occasion.

The voltage regulator was used to differentiate the sources of power between the different components. The 12V channel was used to provide power for the 12V motors for the drive system of the rover, while the 5V channel waws used to provide power for the Raspberry Pi and its components. To go into more detail, the 12V channel is connected to our motor driver, which is then able to provide the necessary power for the 12V motors. On the other hand, the servo motors used within our mechanical systems drew power directly from the Raspberry Pi pins. The camera also was connected directly to the Raspberry Pi main board for both power and data transfer. This component was used to track the lines, intersections, and objects for tasks designated within the scope of this project.

4. Design Analysis

Calculations and simulations were run to roughly verify that the design that the team had come up with would be suitable for the applications presented within the scope of this project. A number of preliminary calculations were performed in the winter quarter reports, which are detailed in this report as well. Then, the relevant calculations regarding the pickup and drop off process for the discs follow those calculations. Finally, after these calculations, a simple FEA study and its results are highlighted, which notes the level of durability expected out of the bottom plate design, and whether it is expected to be sufficient for this project.

4.1. Preliminary Calculations

In order to calculate the drive system power requirements, move profile, and motor torque requirements a total estimate weight of the rover was utilized in the calculations. Each subsystem was weighed separately, taking into account the weight given by SolidWorks and other added components after fabrication. Table 5 represents each subsystem's weight and the estimated total weight of the rover.

Component	Weight (grams)	Quantity	Total Weight
chasis	1739.45	1	1739.45
basket-lift	148.29	1	148.29
arm	119.74	1	119.74
	Total Weight		2007.48

Table 5: Estimated total weight

4.1.1. Drive System Power Requirements

The rover's wheels are made of rubber material and the coefficient of friction between the rubber and the plywood of the track is approximately 0.705.
The required minimum coefficients of friction for the FWD, RWD, and AWD were calculated using the following equation for the respective wheel drive:

$$\mu = \frac{Force\ of\ Weight \times sin(Angle\ of\ Incline)}{Normal\ Force\ of\ Wheels}$$

The values for FWD, RWD, and AWD are 0.4022, 0.8025, and 0.2679 respectively. Figure 11 compares the values of the friction coefficients as the horizontal distance of the center of mass from the rear increases from 0.1 m to 0.195 m (length of the wheelbase).

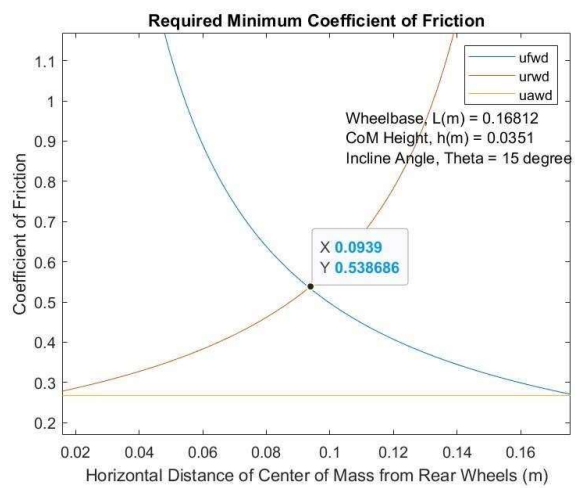

Figure 14: Required Minimum Coefficient of Friction

The normal force between the rear wheels and the ground measured to be 6.3489 and was calculated using the following equation:

$$N_r = \frac{(L - L_c)W_f cos(\theta) + h_c W_f sin(\theta)}{L}$$

where L represents the length of the wheelbase, Lc is the CoM distance from the rear wheels, hc is the CoM height from the ground, Wf is the weight force, and theta is the angle of incline. The normal force acting on the front wheels of the rover was measured to be 12.6669. This force was calculated using the following equation:

$$N_f = \frac{L_c W_f cos(\theta) - h_c W_f sin(\theta)}{L}.$$

The total tractive force equated to 10.19 N by adding the FWD and RWD tractive force. The FWD and RWD tractive force were calculated by multiplying the normal force by the minimum friction coefficient of their respective wheel drive. To determine the capability of the transporter to climb the slope, the force of gravity along the slope is compared to the total tractive force. The force of gravity is 5.09355 Newtons, and the total tractive force exceeds this value as it is double the gravitational force along the slope. The rover will have more than enough tractive force to overcome the resistance of gravity.

4.1.2. Move Profile & Motor Torque Requirements

A maximum and continuous velocity must be established for the segments of the track to perform motor sizing calculations and create a move profile. The range of the velocity the rover will move along the track was 5 to 15 cm/s. The speed of the robot traveling straight will be approximately 10 cm/s and the speed of the robot not traversing straight will be 8 cm/s.

The percent weight distribution between the rear and front wheels was determined using these two equations:

$$\beta_f = \frac{L_c}{L} + \frac{h_c tan(\theta)}{L}$$
$$\beta_r = \frac{L - L_c}{L} + \frac{h_c tan(\theta)}{L}$$

where βf and βr represent the percent weight distribution of the front and rear wheels respectively. βf was determined to be 0.67 and βr was 0.33387. Since the robot will be utilizing front-wheel drive, the coefficient of friction used in the move profile and motor sizing calculations was 0.705, equivalent to the minimum required friction coefficient for FWD previously. The rolling coefficient utilized was 0.003.

The track was divided into 5 segments, as shown in Figure 12 below, and a move profile was created with estimates for average and maximum velocities along each segment.

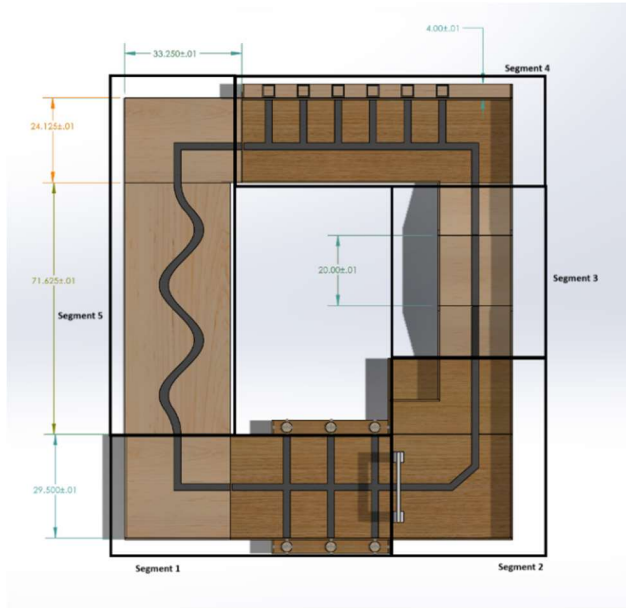

Figure 15: Track Divided into Segments

Table 6 presents the move profile type of each segment along with the acceleration time, constant velocity time, deceleration time, and total time. Table 6 displays the average maximum velocity and acceleration along each segment using the data from table 5 to calculate the values.

Table 6: The estimated amount of time for acceleration, constant velocity, and deceleration of each segment and each segment's move profile type

Path Segment	Move Profile Type	Accel Time (s)	Const Vel Time (s)	Decel Time (s)	Total Time (s)
1	Triangular	25	0	10.62	35.62
2	Trapezoidal	6	8.3	2.39	16.69
3	Trapezoidal	5.95	1	5.95	12.90
4	Triangular	22	0	11.76	33.76
5	Trapezoidal	5	22	8.74	35.74

Table 7: The estimated maximum velocity and acceleration for each segment

Path Segment	d_tot (m)	t_tot (S)	V_max (m/s)	a_max (m/s^2)
1	284.96	35.62	10	0.4
2	133.5	16.69	10	1.666666667
3	129.03	12.90	12	2.016806723
4	270.07	33.76	10	0.4545454545
5	285.94	35.74	10	1.428571429

With the data from tables 6 and 7, a move profile (shown in figure 16) was created with estimates of average and maximum velocities along each segment as the robot traverses the course.

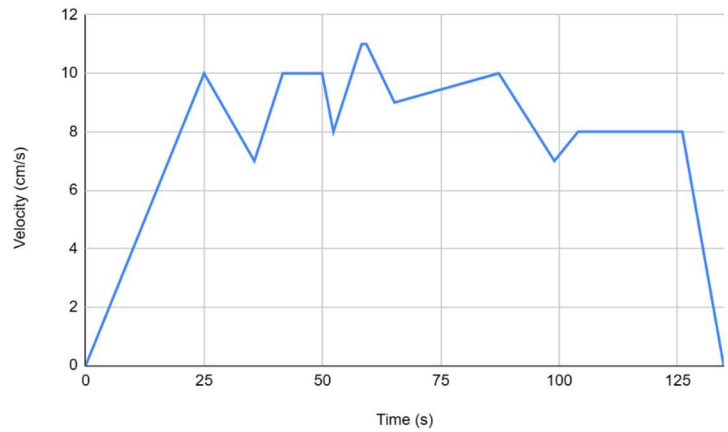

Figure 16: Move Profile

Based on the move profile, the required propulsion force for each segment was calculated by summing the inertia force, weight force, friction force, and rolling friction force. Table 8 displays all the measured values for each respective force.

Table 8: The estimated maximum required propulsion force for each segment (summation of inertia force, weight force, wheel friction force, and rolling force)

Path Segment	F_i (N)	F_w (N)	F_f (N)	F_rol (N)	F_prop (N)
1	0.802992	19.673304	15.78865097	0.059019912	36.32
2	3.3458	19.673304	15.78865097	0.059019912	38.87
3	4.04869916	19.673304	15.25066573	0.05700885727	39.03
4	0.9124909091	19.673304	15.78865097	0.059019912	36.43
5	2.867828571	19.673304	15.78865097	0.059019912	38.39

The required power along each segment was calculated by multiplying the required propulsion and the maximum velocity.

Table 9: The estimated required propulsion power for each segment

Path Segment	P_prop (W)
1	363.24
2	388.67
3	468.36
4	364.33
5	383.89

The required propulsion torque was calculated using the following equation:

$$T_{prop} = \frac{P_{prop} \times \frac{d_w}{2}}{\eta}$$

where Tprop is the required torque propulsion, Pprop is the required power, dw is the wheel diameter, and represents the system's efficiency. Table 10 displays the required torque propulsion values for each segment.

Table 10: Estimated maximum required propulsion power and motor torque along each segment

Path Segment	P_prop (W)	T_prop (N-m)
1	363.2396688	24.32
2	388.6677488	26.02
3	468.356133	31.36
4	364.3346579	24.32
5	383.8880345	24.32

The drive mechanism for our robot is a front-wheel drive as turning is a priority and the required torque for front-wheel drive based on our calculations is 0.2388158 Nm. As long as the robot's motor torque is higher than 0.2388158 Nm, the robot should have no problem traversing the course, including the ramp section.

4.2. Basket Lift Calculations

For the rover design, the lifting platform was a main focal point in the delivery and pick up of disks. In regard to this design, calculations were performed to determine the work needed to lift the platform up. The maximum range of motion was assumed to be two inches, as the height of three stacked disks would be one- and one-half inches. Given some tolerance, the maximum range of motion was increased to two inches. So, in order to displace the disks upwards, the force necessary was assumed to be equal to its weight, which was calculated to be 0.566 Newtons. Then, the maximum work done on the mass in lifting the disks by the required range of motion was calculated as follows.

$$W = Fd = (0.566\,N)(2\,inches) = (0.566\,N)(0.0508\,m) = 0.02875\,N \cdot m$$

The motor torque needed to displace the platform was then estimated to the value above, of 0.02875 N-m. Looking at the manufacturer's specifications for the maximum torque that the motor is capable of handling, the maximum motor torque is set to approximately 1.5 kg-cm, which was then converted to be 0.138 N-m. Comparing the maximum motor torque that the motor can handle to the torque needed to displace the three disks by the required range of motion, the crank and shaft mechanism is capable of lifting the disks with a factor of safety of approximately 4.8, which makes our team confident in the design and its capability to provide the necessary lift motion.

4.3. Scooper Arm Calculations

The first general calculation that was performed was to determine the force needed to pick up each disk from its pickup location. The volume of a single disk was calculated as a cylinder with a diameter of 3.5 inches and a height of 0.5 inches. This volume was then multiplied by the density of plywood to determine the weight of a disk. The density range of wood in general was averaged to determine the density of plywood, which was assumed to be 38 pounds per cubic foot. The static coefficient of friction for wood-on-wood was assumed to be 0.375. Multiplying the static coefficient of friction by the normal force, as determined by the weight of a disk, was used to then determine the force needed to push the disk. These calculations are shown below.

$$V = \pi r^2 h = \pi(3.5)^2(0.5) = 4.81\,in^3 = 0.002784\,ft^3$$
$$m = \rho V = (38\,lb/ft^3)(0.002784\,ft^3) = 0.106\,lb = 0.0481\,kg$$
$$\Sigma F_y = N - mg = N - (0.0481\,kg)(9.8\,m/s^2) = 0$$
$$N = 0.47138\,N$$
$$F_{push} = \mu_s N = 0.375(0.47138) = 0.1767675 = 0.177\,N$$

The next calculation that was performed was to determine the force needed to push a stack of disks onto its drop off location. These calculations were similar to those for pushing one disk. However, the mass was multiplied by a factor of three to account for three disks. Also, the coefficient of static friction was changed to 0.4 to account for wood-to-hard-plastic contact. These calculations are shown below.

$$m_2 = 3m = 0.1443\,kg$$
$$\Sigma F_y = N - mg = N - (0.1443\,kg)(9.8\,m/s^2) = 0$$
$$N = 1.41414\,N$$
$$F_{push} = \mu_s N = 0.4(1.41414) = 0.565656 = 0.566\,N$$

The general calculation was then performed to estimate the amount of force that the servo motor and overall mechanism could generate. The manufacturer's specifications stated a maximum torque of 1.5 kg-cm for the servo motor, which was converted to 0.138 N-m. Then, this torque was divided by the radius of the gear, which was over-approximated to 0.01 meters. This resulted in an estimated maximum force for the servo motor of 13.8 Newtons. As can be seen from the above calculations, the maximum force that is needed to push even three disks is 0.566 Newtons, which is more than twenty times less than the maximum force that the motor can handle. As such, our team is confident that the motor and mechanism is more than capable of scooping the disks.

4.4. Bottom Plate FEA Study

The bottom plate for the rover design underwent a simple FEA/FEM analysis to ensure that the plate could withstand the general weight being applied to it on the top, and to ensure that it would not flex to a point where the design was at risk of failing. For the first FEM study, one end of the plate was held fixed and a load of five Newtons was applied to the other end. The resulting simulation is shown below.

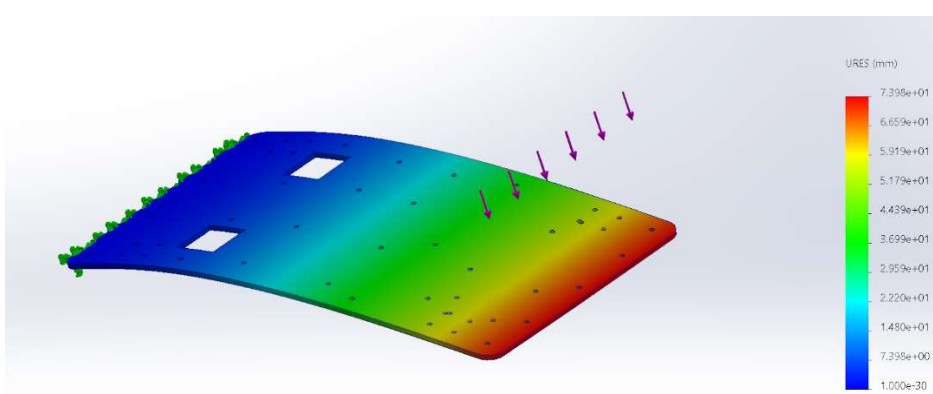

Figure 17: Bending Displacement Study for Bottom Acrylic Plate (Force of 5 Newtons)

With a concentrated load applied to the bottom layer, there is a maximum displacement of approximately 7.2 centimeters. Though this total displacement does raise some concerns in terms of the durability and safety of the design, our team considered that the rover will likely never experience such a concentrated load. Taking this into account, the FEA study was reperformed with an even load, which was meant to simulate the rest of the components being placed on the plate itself. As the load was evenly distributed, the study was shifted from displacement and to a factor of safety analysis. This study is shown below.

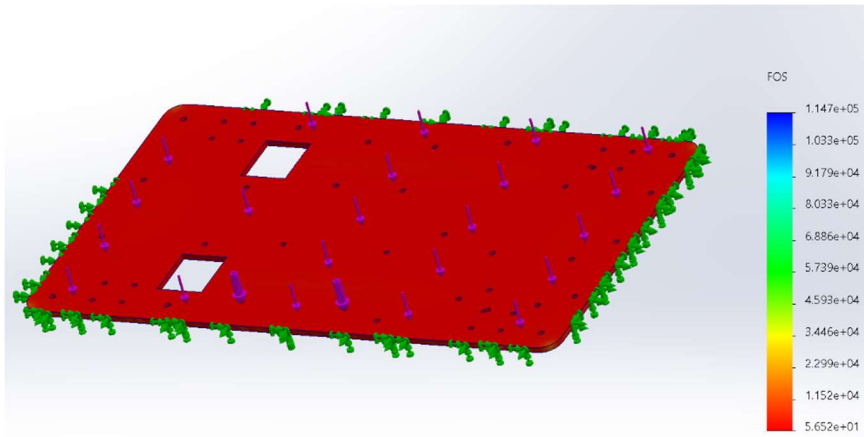

Figure 18: Factor of Safety Analysis for Bottom Acrylic Plate (25 Newton Load)

Based on the results of the FEA study, the minimum factor of safety exceeds fifty for an applied load of 25 Newtons. This gives us confidence that the bottom plate will be able to support the total mass of the rover system without any issues. Even if the applied load is double or triple the simulated amount, the expected factor of safety is still well beyond one, which makes us believe that the overall backbone of the chassis will hold up and ensure no major structural failures.

5. Control System Design

5.1. System Architecture

Figure 19: Communication Network System

The system architecture for the project is centered around a robust integration of hardware and software components to manage and monitor a mobile robot. At the heart of the robot, named `robot_ws` (robot workspace), is a Raspberry Pi which serves as the core mainframe. This Raspberry Pi operates on a Linux kernel and runs Ubuntu 20.04 as its operating system distribution. One of the

key features of this setup is the installation of ROS 2 Humble, a powerful and flexible framework designed specifically for robotics applications.

ROS 2, or Robot Operating System version 2, enhances the capabilities of robotic systems through its modular design and communication tools. It allows for various computational entities, known as nodes, to communicate over a peer-to-peer network. These nodes can publish and subscribe to data streams known as topics, which are crucial for the operation of distributed systems like `robot_ws`. The topics enable different parts of the robotic system to exchange information seamlessly, such as sensor data and control commands, over a network without the need for a direct hardware connection.

To facilitate remote operations and monitoring, `robot_ws` is connected to a personal iPhone T-Mobile hotspot, creating a private network. This network connectivity extends to a laptop, referred to as `dev_ws` (developer workspace), which also runs Linux Ubuntu 20.04 and is equipped with ROS 2. This setup allows for efficient and real-time communication between the robot and the laptop using the same ROS 2 environment. The lack of a physical screen on the Raspberry Pi is circumvented by the ability to stream live data and logs directly to the laptop's terminal over this network, utilizing Wi-Fi to pass topics within ROS 2.

Moreover, the setup includes the use of Secure Shell (SSH) protocol for logging into the Raspberry Pi's terminal from the laptop, allowing commands to be executed remotely. This remote access is crucial for deploying software, debugging, and operational management without the need for direct physical access to the robot. Additionally, the camera feeds from `robot_ws` are viewable on the `dev_ws` through the `rqt_image_view` tool in ROS 2, demonstrating a practical application of the network's capabilities to facilitate remote visual monitoring of the robot's environment, enhancing the user's ability to control and interact with the robot effectively from a distance.

5.2. Closed-Loop Control of Differential Drive Motors with Encoders

For the differential drive robot in this project, driving the motors with precision is critical for achieving accurate navigation and control. This precision is facilitated through the use of encoders and a closed-loop control system, primarily implemented via Proportional-Integral-Derivative (PID) controllers. The Kp, Kd, Ki, and Ko values that were chosen were 20, 12, 0, and 50.

Encoders are crucial components in this setup, serving as the primary means of odometry. They provide real-time feedback by counting the number of increments or steps taken by the motor's output shaft, allowing the system to determine how far and fast each wheel of the differential drive has moved. This feedback is essential for the robot to understand its position and velocity relative to a starting point, effectively allowing it to "know" its location within a given space.

The core of the control system for the motors is the PID controller, which uses the data from the encoders to regulate the motor's speed and position accurately. In a closed-loop system like this, the PID controller continuously adjusts the motor inputs to minimize the error between a desired setpoint and the actual output. The setpoint could be a specific position or velocity, and the actual output is measured by the encoders. The PID controller calculates the error between these values and adjusts the motor control signals accordingly. The proportional component deals with the present error, the integral component sums up past errors (providing correction based on accumulated discrepancies), and the derivative component predicts future errors, offering a damping effect that smooths the response.

Next, we perform the `read` and `write` functions associated with the PID controller to handle encoder feedback and motor commands through the ROS 2 to Arduino bridge. The read function

reads the encoder values to update the position and velocity of each wheel based on the time elapsed since the last read, which is crucial for real-time control adjustments. The write function sends calculated motor control values to the Arduino. This step is critical for applying the PID controller outputs to the motors, adjusting the robot's movement based on the computed velocities.

```
void doPID(SetPointInfo * p) {
  long Perror; // Proportional error
  long output; // Output to motors
  int input;   // Current encoder input

  // Calculate the difference between current and previous encoder values
  input = p->Encoder - p->PrevEnc;
  // Calculate the proportional error
  Perror = p->TargetTicksPerFrame - input;

  // Compute PID output: Proportional, Derivative, and Integral terms
  output = (Kp * Perror - Kd * (input - p->PrevInput) + p->ITerm) / Ko;

  // Update previous encoder value
  p->PrevEnc = p->Encoder;

  // Adjust output based on previous output and check for saturation
  output += p->output;
  if (output >= MAX_PWM)
    output = MAX_PWM;
  else if (output <= -MAX_PWM)
    output = -MAX_PWM;
  else
    p->ITerm += Ki * Perror; // Update integral term if within output limits

  // Set the new output and previous input for next iteration
  p->output = output;
  p->PrevInput = input;
}

hardware_interface::return_type DiffDriveArduino::read() {
  // Calculate time delta since last read
  auto new_time = std::chrono::system_clock::now();
  std::chrono::duration<double> diff = new_time - time_;
  double deltaSeconds = diff.count();
  time_ = new_time;

  // Check connection to Arduino
  if (!arduino_.connected()) {
    return return_type::ERROR;
  }

  // Read encoder values and calculate new positions and velocities
  arduino_.readEncoderValues(l_wheel_.enc, r_wheel_.enc);
  double pos_prev = l_wheel_.pos;
  l_wheel_.pos = l_wheel_.calcEncAngle();
  l_wheel_.vel = (l_wheel_.pos - pos_prev) / deltaSeconds;

  pos_prev = r_wheel_.pos;
  r_wheel_.pos = r_wheel_.calcEncAngle();
```

```
  r_wheel_.vel = (r_wheel_.pos - pos_prev) / deltaSeconds;

  return return_type::OK;
}

hardware_interface::return_type DiffDriveArduino::write() {
  // Ensure Arduino is connected
  if (!arduino_.connected()) {
    return return_type::ERROR;
  }

  // Send motor values adjusted for radians per count and loop rate
  arduino_.setMotorValues(l_wheel_.cmd / l_wheel_.rads_per_count / cfg_.loop_rate,
r_wheel_.cmd / r_wheel_.rads_per_count / cfg_.loop_rate);

  return return_type::OK;
}
```

5.3. Wheel Velocity Calculations and Commands

In the context of ROS (Robot Operating System), the communication of motion commands for robots, particularly differential drive robots, is facilitated using messages of type geometry_msgs/Twist. This message type is specifically designed to convey velocity commands in a 3D space, encapsulating both linear and angular velocities. Here's how this relates to the application of the differential drive formulas:

The geometry_msgs/Twist message consists of two main components, each of type geometry_msgs/Vector3:

- **linear**: Represents linear velocity in free space broken down into its x, y, and z components.
- **angular**: Represents angular velocity in free space broken down into its x, y, and z components.

For a differential drive robot, typically, only the x-component of the linear velocity (linear.x) and the z-component of the angular velocity (angular.z) are used:

- **linear.x**: This is the forward linear velocity of the robot along the x-axis, which corresponds to the robot moving straight ahead or backward.
- **angular.z**: This represents the angular velocity around the z-axis (upward), indicating how fast the robot turns to the left or right.

Using the provided linear and angular velocities, along with the robot's physical parameters (wheel separation and radius), it calculates the required speed for each wheel. The formula used is:

vleft=vlinear+ wheel separation2wheel radius
vright=vlinear - wheel separation2wheel radius

Where:

- vlinear is taken from linear.x of the Twist message.
- is taken from angular.z of the Twist message.

ROS and Twist messages in a typical ROS system (particularly our system):

- **Publishers**: Nodes that control the robot or simulate its behavior will publish Twist messages to topics such as /cmd_vel, which are subscribed to by the robot's drive controller
- **Subscribers**: The differential drive controller node subscribes to the /cmd_vel topic, receives Twist messages, and calculates the required wheel speeds based on the formulas above.

5.4. Camera System

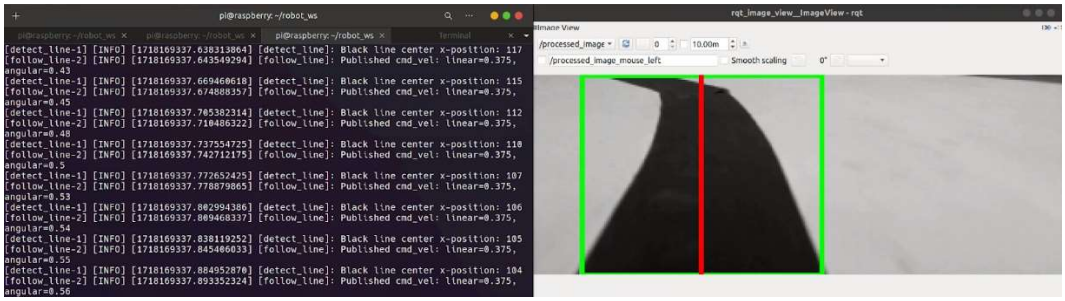

Figure 20: Incorporating Computer Vision with Motor Steering Control

The camera system for the robot incorporates a sophisticated line detection and following mechanism using ROS 2 (Robot Operating System), OpenCV (Open Source Computer Vision Library), and the CV Bridge. This section of the report discusses the implementation details, focusing on the software components and their interactions.

The camera system's primary objective is to detect a line on the floor and adjust the robot's trajectory to follow this line using visual cues. The system is split into two main ROS 2 nodes: `LineDetector` and `FollowLine`.

The `LineDetector` node is responsible for processing incoming video frames from a camera, detecting the presence and position of a line, and publishing this information.

Key Components:

1. **Image Subscription and Publication:**
   - The node subscribes to `/image_in` for raw image data and publishes processed images for visualization to `/original_image` and `/processed_image`.
2. **Image Processing:**
   - Converts the ROS image message to an OpenCV format.
   - Transforms the image to grayscale to simplify processing.
   - Resizes and crops the image to focus processing on the relevant area where the line is expected.
   - Applies a binary threshold to isolate the line based on color intensity.
3. **Line Detection:**
   - Uses morphological operations to clean up the image.
   - Finds contours and identifies the largest one, presumed to be the line.
   - Calculates the horizontal center of the line and publishes this as a `Point` message to `/line_position`.
4. **Green Bounding Box and Red Center Line (Please See Above Figure):**
   - The core of the line detection is identifying the largest contour in the processed image, which presumably represents the line.
   - Green Bounding Box: A rectangle is drawn around this contour. This box helps in visually identifying the detected line on the screen and serves as a reference for the detected line's width and position.
   - Red Center Line: A vertical line is drawn at the center of this bounding box. This line represents the center of the detected line, which is crucial for calculating the deviation from the center of the robot's path.

5. **Handling Different Scenarios**
    - The robot's response is adapted based on various line detection scenarios, each recognized by specific characteristics of the detected bounding box. The value passed is an integer to separate different cases and handle them appropriately.
    - Ordinary Line Following (x>=0): When a regular line is detected without touching the image's edges, the center x-coordinate of the bounding box is passed to the followline node. The robot maintains a steady forward movement while making minor adjustments to stay aligned with the line.
    - Sharp Turn (x=-4 [left] or x=-3 [right]: Recognized when the bounding box touches either the left or right edge of the image. Depending on which side is touched, a sharp turn command is issued to steer the robot accordingly.
    - Intersection (x=-2): Detected when the bounding box touches both the left and right edges of the image, indicating an intersection. Here, the robot might need to stop or choose a direction based on additional logic or commands.
    - Seeing Nothing (x=-1): When no significant contours are detected, it's presumed that there is no line in the visible area. The robot may stop or perform a search maneuver to relocate the line.

The `FollowLine` node receives the line position and generates velocity commands to steer the robot accordingly.

Functionality:

1. **Line Position Subscription**:
    - Subscribes to `/line_position` to receive updates on the line's position.
2. **Velocity Command Computation**:
    - If no line is detected (x-position is -1), the robot moves slowly backward in case it missed something.
    - If a line is detected (x-position >= 0), it calculates the error between the line's position and the image center and implements a proportional control strategy to adjust the robot's heading based on this error.
    - If an intersection is detected (x-position =-2), it makes a complete stop and shutdowns the node to make the next appropriate step (i.e. perform deadreckoned velocity commands in the pick-up/drop-off zone to reach the item)
    - If a sharp turn is detected (x-position = -3 or -4), it makes a higher angular sweep to match the turn.
3. **Command Publication**:
    - Publishes the computed linear and angular velocities to `/cmd_vel`, which would typically be subscribed to by the robot's mobility control system.

Both nodes operate within the ROS 2 ecosystem, leveraging its robust messaging infrastructure for inter-node communication. Testing can be conducted using `rqt_image_view` to visualize the input and output images alongside real-time adjustments to robot movement based on the line detection outputs.

detect_line.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Point
```

```
from cv_bridge import CvBridge, CvBridgeError
import cv2
import numpy as np
from std_msgs.msg import Int32

class LineDetector(Node):
    def __init__(self):
        super().__init__('line_detector')
        self.get_logger().info('Line Detector Initialized')
        self.image_sub = self.create_subscription(Image, "/image_in", self.image_callback,
10)
        self.point_pub = self.create_publisher(Point, "/line_position", 10)
        self.processed_image_pub = self.create_publisher(Image, "/processed_image", 1)
        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Int32,
            'shutdown_signal',
            self.shutdown_callback,
            10)

    def shutdown_callback(self, msg):
        if msg.data == 1:  # Check if the shutdown signal is received
            self.get_logger().info('Shutdown signal received, shutting down...')
            line_detector.destroy_node()
            rclpy.shutdown()

    def find_line(self, cv_image, draw_image):
        black_threshold = 50
        _, black_mask = cv2.threshold(cv_image, black_threshold, 255, cv2.THRESH_BINARY_INV)
        black_mask = np.uint8(black_mask)

        kernel = np.ones((5, 5), np.uint8)
        black_mask = cv2.morphologyEx(black_mask, cv2.MORPH_OPEN, kernel)
        black_mask = cv2.morphologyEx(black_mask, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(black_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        if contours:
            largest_contour = max(contours, key=cv2.contourArea)
            x, y, w, h = cv2.boundingRect(largest_contour)
            line_center_x = x + w // 2

            # Draw bounding box in green
            cv2.rectangle(draw_image, (x, y), (x + w, y + h), (0, 255, 0), 2)

            # Draw center line in red
            cv2.line(draw_image, (line_center_x, y), (line_center_x, y + h), (0, 0, 255), 2)

            # Check if the bounding box touches the image edges
            img_width = draw_image.shape[1]
            img_height = draw_image.shape[0]

            if x == 0 and (x + w) == img_width and y > img_height // 4:
                return -2, black_mask  # Intersection
            elif x == 0 and y > 0 and y < img_height // 2:
                return -4, black_mask  # Sharp left turn
            elif (x + w) == img_width and y > 0 and y < img_height // 2:
                return -3, black_mask  # Sharp right turn
            # elif x == 0 and (x + w) == img_width and y == 0 and (y + h) == img_height:  #
All black
            #     return -5
```

```
            return line_center_x, black_mask  # Normal line detection

        return -1, black_mask  # No line detected (empty space or obstacle)


    def image_callback(self, data):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(data, "bgr8")
        except CvBridgeError as e:
            self.get_logger().error(str(e))
            return

        gray_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)
        scale_percent = 50
        width = int(gray_image.shape[1] * scale_percent / 100)
        height = int(gray_image.shape[0] * scale_percent / 100)
        dim = (width, height)
        resized_gray_image = cv2.resize(gray_image, dim, interpolation=cv2.INTER_AREA)

        crop_start = height // 2
        cropped_image = resized_gray_image[crop_start:height, :]

        # Create a copy for drawing
        drawing_image = cv2.cvtColor(cropped_image, cv2.COLOR_GRAY2BGR)

        # Pass the drawing image for modifications
        line_x, processed_image = self.find_line(cropped_image, drawing_image)

        processed_img_msg = self.bridge.cv2_to_imgmsg(drawing_image, "bgr8")
        self.processed_image_pub.publish(processed_img_msg)

        self.get_logger().info(f"Black line center x-position: {line_x}")
        point_out = Point(x=float(line_x), y=0.0, z=0.0)
        self.point_pub.publish(point_out)


def main(args=None):
    rclpy.init(args=args)
    line_detector = LineDetector()
    rclpy.spin(line_detector)

    line_detector.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

follow_line.py

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point, Twist
# from geometry_msgs.msg import TwistStamped
# from std_msgs.msg import Header
from std_msgs.msg import Int32
# import time

class FollowLine(Node):
    def __init__(self):
        super().__init__('follow_line')
```

```
        self.subscription = self.create_subscription(
            Point,
            '/line_position',
            self.line_callback,
            10)
        # self.declare_parameter("base_link", "")
        # self.frame_id = str(self.get_parameter("base_link").value)
        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)
        # self.publisher_ = self.create_publisher(TwistStamped, '/cmd_vel_tracker', 10)
        self.get_logger().info('Follow Line Node Started')
        self.shutdown_publisher = self.create_publisher(Int32, 'shutdown_signal', 10)
        # self.timer = self.create_timer(5, self.publish_signal)  # Publish every 5 seconds

    def line_callback(self, msg):
        twist = Twist()
        # commands = TwistStamped()
        center_x = msg.x

        if center_x >= 0:
            image_center_x = 160  # Assuming image width of 640
            error = center_x - image_center_x
            twist.linear.x = 0.375  # Constant forward speed
            twist.angular.z = -0.01 * error  # Proportional control
        elif center_x == -4:  # sharp left turn
            twist.linear.x = 0.5
            twist.angular.z = 2.5
        elif center_x == -3:  # sharp right turn
            twist.linear.x = 0.5
            twist.angular.z = -2.5
        elif center_x == -2:  # intersection
            twist.linear.x = 0.0
            twist.angular.z = 0.0
            self.get_logger().info('Intersection detected, shutting down node.')
            msg = Int32()
            msg.data = 1
            self.shutdown_publisher.publish(msg)
            follow_line.destroy_node()
            rclpy.shutdown()
        # elif center_x == -5:  # all black so at pickup/dropoff
            # subprocess.run(['sudo', '/home/pi/pickup.sh'], check=True)
        elif center_x == -1:  # No line detected (no box)
            twist.linear.x = -0.3
            twist.angular.z = 0.0
            # commands.header.stamp = self.get_clock().now().to_msg()
            # commands.header.frame_id = self.frame_id
            # commands.twist.linear.x = 0.5
            # commands.twist.angular.z = 4.0
            # self.publisher_.publish(twist)
            # self.get_logger().info(f'Published cmd_vel: linear={commands.twist.linear.x},
angular={commands.twist.angular.z}')

        self.publisher_.publish(twist)
        self.get_logger().info(f'Published cmd_vel: linear={twist.linear.x},
angular={twist.angular.z}')


def main(args=None):
    rclpy.init(args=args)
    follow_line = FollowLine()
    rclpy.spin(follow_line)
    follow_line.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

5.5. Arm Mechanism

The capstone project incorporates a sophisticated arm mechanism, controlled through Bash scripts interfacing directly with the GPIO pins on a Raspberry Pi. This setup is necessary due to compatibility issues with a Python package and the requirement for administrative privileges in hardware control. The arm mechanism operates using three servos: two positional and one continuous, all managed via PWM signals ranging from 500 to 2500 microseconds.

- **Positional Servos**: These servos are relatively straightforward to control, as their position correlates directly with the PWM signal value. Adjusting the pulse width sets the angle of the servo arm, providing precise control over its movement.
- **Continuous Servo**: This servo type operates differently, as it rotates continuously at a speed determined by the PWM signal. Controlling this servo involves dynamically adjusting the PWM values to change the speed and setting it to a neutral position to stop the movement.

Due to specific project constraints, including security measures that necessitate superuser privileges, the servo control logic is implemented using Bash scripts. These scripts command the `pigpiod` daemon to send accurate PWM signals to the GPIO pins, ensuring precise manipulation of the servos.

The control scripts are designed to perform complex sequences for tasks such as picking up and dropping off objects:

- **Pickup and Dropoff Scripts**: These scripts sequentially activate the servos to execute tasks involving multiple steps, such as gripping, moving, and releasing objects. Each action is timed using `sleep` commands to allow movements to complete before proceeding to the next step.
- **Nested Bash Script Control Logic**: The `pickupzone.sh` and `dropoffzone.sh` script utilizes a creative approach to manage movement sequences based on user input. It processes predefined commands linked to specific input numbers and executes corresponding actions if those numbers are present in the input arguments.

An additional script enhances the control scheme by using letters to specify the number of repetitions for certain commands:

- **Letter-to-Movement Mapping**: The script assigns letters ('a' to 'f') to dictate the number of times a "forward" command is executed. This approach allows for a nuanced control mechanism, adaptable to complex operational requirements.
- **Execution Strategy**: The script segregates inputs of pick-up and drop-off zones into numbers and letters, processes them accordingly, and executes the "forward", “left”, or “right” movement via a simple logic algorithm.

pickupzone.sh

```
#!/bin/bash

function left() {
    ./left.sh
}
```

```
function right() {
    ./right.sh
}

function forward() {
    ./forward.sh
}

# Input numbers from command line arguments
input_numbers=("$@")

# Define sequence and associated numbers
commands=(
    "left 1"
    "right 2"
    "forward"
    "left 3"
    "right 4"
    "forward"
    "left 5"
    "right 6"
    "forward"
)

# Process commands for numbers
for cmd in "${commands[@]}"; do
    if [[ "$cmd" =~ " " ]]; then
        action=${cmd%% *}   # Extract the action (before the space)
        number=${cmd##* }   # Extract the number (after the space)
        if [[ " ${input_numbers[*]} " =~ " ${number} " ]]; then
            $action  # Call the function by its name
        fi
    else
        # It's just a 'forward' command, so call it directly
        $cmd
    fi
done
```

dropoffzone.sh

```
#!/bin/bash

# Function to execute a command multiple times based on letter input
function execute_multiple_times() {
    local command_script=$1
    local times=$2
    for ((i=0; i<times; i++)); do
        ./$command_script
    done
}

# Mapping letters to the number of times a command should be executed
declare -A letter_to_number=( ["a"]=1 ["b"]=2 ["c"]=3 ["d"]=4 ["e"]=5 ["f"]=6 )

# Input numbers and letters from command line arguments
input_numbers=()
input_letters=()

# Separate numbers and letters into different arrays
for arg in "$@"; do
    if [[ "$arg" =~ [a-f] ]]; then
        input_letters+=("$arg")
```

```
    else
        input_numbers+=("$arg")
    fi
done

# Finally, process commands for letters
for letter in "${input_letters[@]}"; do
    if [[ -n "${letter_to_number[$letter]}" ]]; then
        execute_multiple_times "forward1.sh" "${letter_to_number[$letter]}"
    fi
done

./right1.sh
```

pickup.sh

```
#!/bin/bash
sudo pigpiod

# initial
pigs s 14 1480
sleep 0.5
pigs s 15 515
sleep 0.5
pigs s 18 500
sleep 0.5

# open
pigs s 15 700
sleep 0.5

# extend
pigs s 14 2500
sleep 2.5
pigs s 14 1480
sleep 0.5

# close
pigs s 15 515
sleep 0.5

# pullback
pigs s 14 500
sleep 2.5
pigs s 14 1480
sleep 0.5

sudo killall pigpiod
# chmod +x script.sh to make this text file executable and run ./script.sh
```

dropoff.sh

```
#!/bin/bash
sudo pigpiod

# initial
pigs s 14 1480
sleep 0.5
pigs s 15 505
sleep 0.5
pigs s 18 500
sleep 0.5
```

```
# raise
pigs s 18 2500
sleep 1

# open
pigs s 15 700
sleep 0.5

# extend
pigs s 14 2500
sleep 2.5
pigs s 14 1480
sleep 0.5

# pullback
pigs s 14 500
sleep 2.5
pigs s 14 1480
sleep 0.5

# close
pigs s 15 505
sleep 0.5

# dropdown
pigs s 18 500
sleep 0.5

sudo killall pigpiod
# chmod +x script.sh to make this text file executable and run ./script.sh
```

5.6. State Diagram

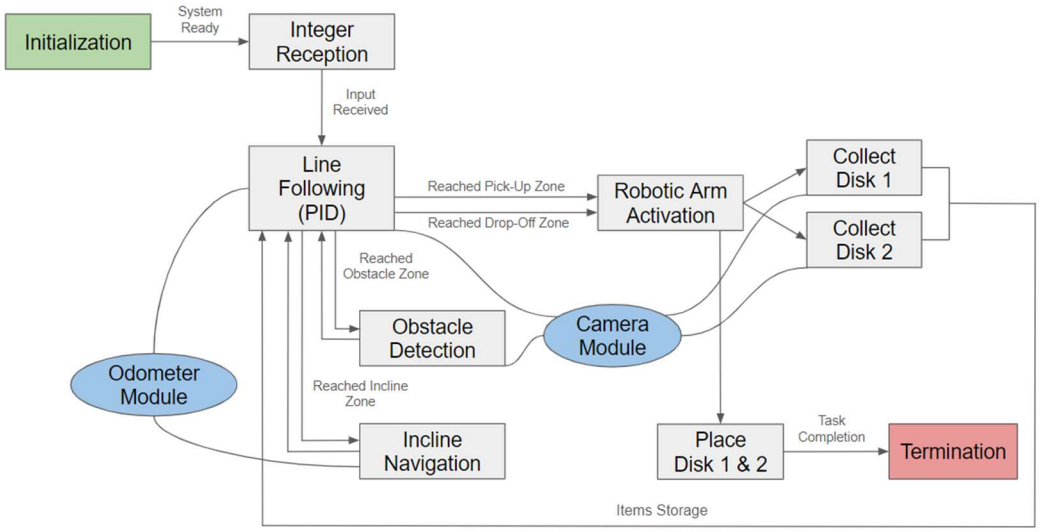

Figure 21: State Diagram of Rover Robot Design

States (Nodes):

1. **Initialization:** The rover initializes its subsystems and sensors to prepare for the upcoming tasks.
2. **Integer Reception:** The system waits to receive an integer input that will determine its target drop-off location.

3. **Line Following (PID):** Utilizing PID control, the rover follows a black line to navigate to different zones. This state likely employs both the camera module for visual line tracking and the odometer module for precise movement control.
4. **Robotic Arm Activation:** Upon reaching the pick-up zone, the rover activates its robotic arm to collect disk objects. The camera module may assist in object recognition and positioning of the arm.
5. **Collect Disk 1:** The robotic arm collects the first disk object at the specified spot within the pick-up zone.
6. **Collect Disk 2:** After collecting the first disk, the rover collects the second disk object.
7. **Obstacle Detection:** The camera module identifies obstacles on the path, and the system decides the appropriate action to take.
8. **Incline Navigation:** The IMU detects inclines or declines in the path, and the rover adjusts its motor power. The odometer module helps in adjusting the movement based on the slope detected by the IMU.
9. **Place Disk 1 & 2:** The rover places the collected disks at the designated drop-off zone. The camera module may be used to ensure precise placement.
10. **Termination:** The mission is completed, and the rover shuts down its operations or prepares for another run.

Actions (Arrows):

1. **System Ready:** Transition from Initialization to Integer Reception, indicating the rover is ready for input.
2. **Input Received:** Transition from Integer Reception to Line Following (PID), indicating the start of the navigation process.
3. **Reached Pick-Up Zone:** Transition from Line Following (PID) to Robotic Arm Activation, signaling arrival at the collection area.
4. **Reached Obstacle Zone:** Transition indicating the Line Following state has encountered an obstacle and must engage Obstacle Detection.
5. **Reached Drop-Off Zone:** Transition from Line Following (PID) directly to Place Disk 1 & 2, indicating the rover has reached the location where it should deliver the collected disks.
6. **Reached Incline Zone:** Transition from Line Following (PID) to Incline Navigation, signaling the presence of a slope that requires careful navigation.
7. **Task Completion:** Transition from Place Disk 1 & 2 to Termination, indicating that the rover's tasks have been completed successfully.

6. Product Fabrication

The design and fabrication process for the rover was largely split into three parts, which involved the main chassis, the basket-lift, and the scooper arm. The general fabrication process for these three subsystems is divided and described below.

6.1. Chassis Fabrication

The main design of the chassis revolved around flat layers made from acrylic sheets held apart by standoffs in order to provide the main body for the rover. The flat layers were originally designed to be fabricated using a 3D printer and made from PLA, as the general rigidity of the material was seen to be favorable in terms of strength and the overall use of a 3D printer allowed for rapid prototyping in the case that the dimensions within the design were incorrect. However, the size of the layers within the chassis design were too large in terms of the total length of the layers. They proved to be too long

to fit onto the print beds for the 3D printers offered within the Boelter Makerspace, which called for either a change in design or fabrication choice. The latter was chosen, so the layers were fabricated using 3-mm thick acrylic sheets that were laser cut.

The standoffs in between the layers were fabricated using a 3D printer, as the design relied on a consistent platform height to operate. In the case that the platform height varied from the correct height that the design needed to pick up and drop off the disks, the standoffs could be quickly reprinted with variable heights to account for this. The standoffs were connected using M3 screws that would thread into the standoffs. On another note, the motors and their corresponding wheels were ordered. Attaching these motors was simple, as the product came with motor mounts that could be screwed onto the bottom layer using M2.5 screws. The back two caster wheels were also ordered. Due to their smaller size than the 65-mm front wheels, standoffs were 3D printed using PLA to make up for this height difference.

6.2. Basket Lift Fabrication

All components of the basket-lift were 3D printed from PLA, except for the ordered positional servo motor. The platform for the basket lift underwent multiple iterations to achieve the correct dimensions, allowing a tight fit while enabling smooth vertical movement along the basket's rim. The expansion of PLA posed challenges in achieving precise dimensions. The standoffs for the basket-lift feature two holes: one for mounting onto the chassis using screws and nuts, and another on top for securing the basket's outside rim with M3 screws. Assembly of the crank, connecting rod, and platform involved two M2.5 screws and lock nuts. The lock nuts allow for rotation without loosening during operation. The servo motor is mounted on a motor mount using M2 screws, which are then connected to the standoffs. The standoffs and servo motor mount are all secured with M3 screws and nuts attached to the chassis.

6.3. Scooper Arm Fabrication

All the manufactured components for the scooper arm design were fabricated using a 3D printer. Each part was made from PLA at the default settings given from the Boelter Makerspace. The arm's base was mounted to the top-most layer of the chassis using M3 screws, and the mount for the rack and pinion design was also mounted to the base using M3 screws. The main perimeter of the scooper was attached to the rack using two M2.5 screws and appropriately sized bolts. The continuous servo motor used for the rack and pinion was ordered online and mounted using M2 screws. The continuous servo motor used for the scooper's rotational degree of freedom was also order and mounted using M2 screws. The rotating part of the scooper and the gear for the rack and pinion design were attached to the servo by creating a fixture that was glued into each part. This allowed for the part to attach to the servo motor via this fixture, and an additional screw was used to secure each part to the appropriate servo motor.

6.4. Final Product

The separate subsystems were combined into a single, holistic rover that managed to have a degree of functionality. A picture of the completed rover following the drop test during the day of the final competition is shown below.

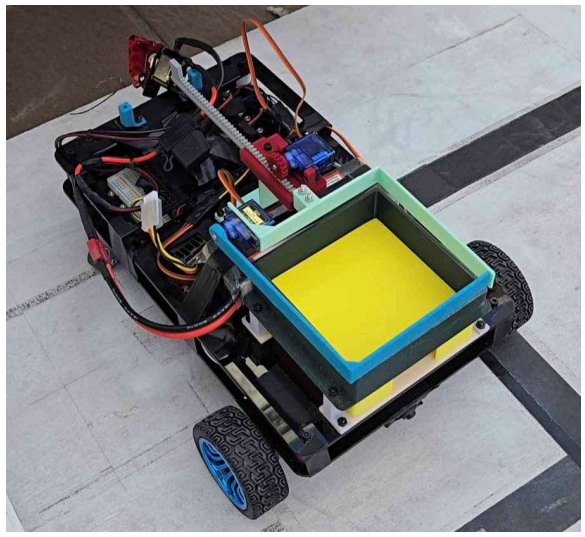

Figure 22: Overhead View of Fully Assembled Rover Design

The overall design of the rover remained the same as the CAD model that it was built around, with the main chassis, basket-lift, and arm subsystems closely resembling the design. The positions for the main electronic components also remained the same. However, due to issues with accessing cables and with cable management, the physical design does appear more disorganized than before. This is a major learning point from this project, in that integrating electrical wiring and cable management into designs is also a critical component of the design process. Being able to add in features to mitigate this would be a future endeavor, if there is ever the opportunity to expand upon this project. The addition of a cover of box would also have hidden the cable and increased the aesthetic value of the robot, which is also a future consideration.

7. Product Performance Testing and Evaluation

One significant problem encountered during field testing was that the elevated platforms for pick-up and drop-off were approximately 1 cm lower than expected. To compensate, the standoffs connecting the upper and lower acrylic layers of the chassis had to be shortened by 1 cm, reducing the space available for electronics between the layers. As a result, the battery had to be relocated on top of the chassis, and an extra hole was cut in the top layer to accommodate protruding electronics. This issue was related to the ongoing height inconsistency problem. Despite lowering the entire height of the rover, the inconsistent platform heights caused pick-up and drop-off failures at certain stations. To mitigate this, the outside rim of the basket was sanded down by roughly 2 mm.

During the development of the autonomous section, debugging in Python presented challenges due to the use of multiple imported libraries, which prolonged the troubleshooting process. For instance, adjusting the motor speeds to allow sufficient time for the camera to process data and perform error corrections proved difficult. In addition, the PID controller parameters were initially tuned for high speeds and resetting these parameters for lower speeds required considerable effort. Determining the specific functions of various code segments was a complex and time-consuming process, delaying performance testing.

Another issue encountered during testing and evaluation was the camera's inconsistency after recharging the battery and rebooting the rover. Each time the rover was powered off, the Raspberry Pi's memory was wiped, necessitating the reupload of the code upon the reboot. Consequently, the camera required a "warm-up" period of around 15 minutes before operating smoothly, leading to

inconsistent trials and extended performance testing times. When the camera operated smoothly, line tracking was nearly perfect, and the rover could navigate the entire course without failure.

7.1. Runtimes

The rover was able to traverse up and down the ramp and follow the curved line within approximately 14 to 16 seconds each. The pick-up section took around 1 to 2 minutes and 30 seconds, depending on the order of the rice roll and the camera's data processing speed. The drop-off section took approximately 30 seconds to 2 minutes, also depending on the dice roll and the distance to the correct drop-off station. The individual process for picking up one disk takes around 3 to 4 seconds while the process for unloading is around 5 to 6 seconds.

7.2. Overall Performance

Overall, the rover demonstrated reliable performance once initial issues were addressed. The adjustments to the chassis and basket-lift improved the consistency of disk pick-up and drop-off. Debugging and tuning the autonomous code enhanced the rover's navigation and data processing capabilities. Despite the camera's initial warm-up requirement, the rover's line tracking and course navigation were effective, achieving the desired outcomes during testing. Relying on the camera's performance, the rover can complete the course with accurate disk handling and efficient navigation.

8. Work Breakdown Schedule

8.1. Winter Quarter

The first three weeks of the winter quarter were spent coming up with design concepts in the form of sketches and rough CAD designs that showed generally how each design could theoretically function to perform the task at hand. Also, the first couple of weeks were spent delegating tasks to each member of the team, as well as gaining access to the SolidWorks software needed to design the rover. From these, the next couple of weeks were spent creating a general design that functioned as the central design for the rover. By week five, the team used the dimensions and masses from this initial design to prepare motor sizing calculations that would be used to determine what type of motor would be needed for our design. Based off of these initial designs, the team ordered 12V motors and began to develop a functioning drive system.

Over the next couple of weeks, the design team attempted to revise the initial design to be best optimized for the provided venue CAD. The design team saw a reassignment of members around this time, while the electronics team continued working on the drive system, which was generally operational by week eight. By week eight, a conceptual design report was created based on this initial design, while the new design team started from the concept designs to create a new design. By the end of the winter quarter, the team put together a preliminary design report based on the initial design laid out in Section 3 of this report. However, that design was entirely scrapped during spring break and the creation of a new CAD design was started. The goal was set to finish early in the spring quarter with a focus on the existing and functioning drive system, such that the rover could be assembled and tested as soon as possible.

8.2. Spring Quarter

The first two weeks of the spring quarter involved redesigning the rover from scratch. With a reassignment in tasks, the new team members created a layered rover design largely based on the kit

provided during lab from winter quarter, as well as on the electronic components needed for the drive system. The design was largely finalized midway through week three, and the focus of the group shifted to finding the appropriate parts online and ordering them from the correct vendors. As parts were ordered or reordered throughout the project timespan, the bill of materials was updated to reflect these changes. As the parts began to arrive by weeks four and five, the fabrication process was started. During weeks four and five, the parts needed for the chassis were laser cut and 3D printed. Then, starting from week five, the printing process for the parts needed for the other two subsystems was started. However, due to the closure of the Boelter Makerspace in week six and the need to readjust the designs slightly to account for tolerance issues while printing, the fabrication process saw significant delays. By week nine, the entire rover was largely assembled and ready for testing. Throughout the timespan from week nine to the final competition date, the controls team worked through getting the drive system to work autonomously and to tackle the tasks of delivering the disks.

Due to the closure of the Boelter Makerspace in week six and the need to readjust for tolerance issues, the fabrication process saw significant delays. However, by week nine the entire rover was largely assembly and ready for testing. Throughout the end of week nine to the final competition date, the controls team largely worked through getting the drive system to work autonomously and to tackle the tasks of delivering the disks. While testing the separate tasks (i.e. pick up, drop off, line tracking, going up the hill, object detection), the team saw significant progress and consistent results. However, there were a lot of complications involved with combining the tasks. Many of these problems were resolved by Tuesday of finals week, though the design saw a lot of issues in regards to the camera being unable to process data quick enough to handle all the tasks. Aside from that, the rover experienced issues with flexures in certain 3D printed parts and the camera breaking during an accident while testing during Tuesday of finals week. These issues were repaired such that the rover was still functional, but a drop in performance following this was noted. By the final competition date, the rover managed to loosely track the curved line, pick up three disks, go up the ramp with some difficulty in following the line, and drop off the disks at the wrong location within different run periods.

9. Bill of Materials and Cost Analysis

Item No.	Item Name	Vendor	Qty	Unit Cost	Total Cost
1	Raspberry Pi	Owned	1	$0.00	$0.00
2	Raspberry Pi Camera	Amazon	2	$20.75	$41.50
3	Raspberry Pi Camera Extension Ribbon Cables	Amazon	1	$10.94	$10.94
4	12V DC Motor with Encoders and Wheels (130 RPM)	Amazon	2	$17.08	$34.16
5	Caster Wheels (Set of 4)	Amazon	1	$9.99	$9.99
6	Acrylic Sheets (Set of 4)	Amazon	1	$14.99	$14.99
7	Continuous Servo Motors (FS90R, Set of 5)	Amazon	1	$27.04	$27.04
8	Positional Servo Motors (SG90, Set of 10)	Amazon	1	$18.77	$18.77
9	Motor Driver	Amazon	1	$6.99	$6.99
10	Arduino Nano	Amazon	1	$11.99	$11.99
11	Voltage Regulator (12V to 5V)	DFRobot	1	$8.50	$8.50
12	12V LiPo Battery	Hobby King	1	$13.98	$13.98
13	12V LiPo Battery Charger	Banggood	1	$31.99	$31.99
14	10 Amp Safety Fuse	Amazon	1	$12.99	$12.99
15	Main Switch, Momentary Switch	Amazon	1	$4.19	$4.19
16	Terminal Strip	Amazon	1	$9.99	$9.99
17	M2 Screw Kit	Amazon	1	$9.99	$9.99
18	M2.5 Screw Kit	Amazon	1	$9.99	$9.99
19	M3 Screw Kit	Amazon	1	$8.99	$8.99
20	2 Channel Dupont Wires	Amazon	1	$7.98	$7.98
21	Sandpaper (Various Grits)	Amazon	1	$4.99	$4.99
Total Number of Parts			**23**	**Total Cost**	**$299.95**

Figure 23: Bill of Materials for Final Rover Design

The bill of materials for all the purchases and orders made within the scope of this project is given in the figure above. All the components listed in the figure were used for the final design, but not every unit was used. To reiterate this point, all twenty-one items differentiated by the item number were used, but not every individual unit within each item number was used. It is to be noted that the Raspberry Pi used within this design is listed for a unit cost of zero dollars because the component used in this project was personally owned. Had this component been purchased, it would have increased our total costs by approximately $100. Also, the Raspberry Pi Camera was listed twice because the first iteration of the camera was stolen in the Boelter Makerspace, which required it to be repurchased. In general, the total cost of the rover project was just under $300, even accounting for this repurchase. Even accounting for the Raspberry Pi, the cost would have been $400. Both values are much lower than the maximum value of $500 allotted to each team.

10. Design Requirement Satisfaction

The design requirements for the rover were split into two levels: high-level design requirements from the project guidelines and low-level design requirements from the project team. The following section discusses whether the rover met these requirements.

10.1. High-Level Design Requirements

Our rover met high-level design requirements 1, 2, and 6 perfectly. The rover fits within the size dimension requirements with dimensions of 29.972 x 24.143 x 16.417 cm. It functions perfectly when dropped from a height of 6 inches and can navigate a 15-degree ramp with a total of 3 disks, even at the lowest motor speed.

Design requirements 3, 5, 7, and 8 were met satisfactorily. For requirements 3, 5, and 7, the performance heavily depended on the Raspberry Pi camera's smooth operation. When the camera lagged, data processing slowed, and the images were captured with a delay, causing the rover to struggle with line-following and dynamic obstacle avoidance. However, when the camera operated smoothly, positional errors were recalculated accurately, and the rover performed well. The camera consistently performed object detection using OpenCV, but slow processing could result in the rover running into obstacles

For requirement 8, the overall product looks marketable, but there is room for improvement in cable management and securing electronics. Many zip ties and tape were used to prevent wires from disconnecting and to stabilize the electronics.

For requirement 4, the rover performed pick-up operations smoothly and drop-off operations satisfactorily. The drop-off depended on the height consistency of the elevated platform. The rover can pick up disks in order but cannot go backward (e.g., picking up 5 first and then going back to pick up 3 and 2).

10.2. Low-Level Design Requirements

For requirements 2, 3, and 6, the rover meets these perfectly. The rover's dimensions are 29.972 x 24.143 x 16.417 cm, and it features a modular design split into three subsections. The arm and basket-lift can be easily removed for repairs and changes, and the top acrylic layer of the chassis can be unscrewed for easy rewiring or electronic part enhancement. The rover also has the lowest possible center of mass.

For requirement 5, performance depends on the camera's operation. If the camera does not lag, the rover can complete the task in less than 5 minutes due to efficient data processing and fewer

positional corrections. If the camera lags, the task takes well over 5 minutes. For requirement 4, the rover does not meet the 5 mm tolerance on all sides, as the length of the rover is almost equivalent to the length limit.

For requirement 1, the rover has the fewest number of DOFs and motors for the current design. This could be reduced further with a different design, such as a 4-bar lift and 1 claw, which would require 4 motors and 2 DOFs compared to the current 5 motors and 3 DOFs for pick-up and drop-off.

11. Summary and Conclusions

The goal of the project was to assemble a rover capable of autonomously picking up and unloading disks in a specified order. During the winter quarter, our project team spent the first half deciding on the design concept and performing calculations with chosen components in mind. We also began developing the drive system and manufacturing components we were certain to use. In the spring quarter, we finalized the design, manufactured, assembled, and tested the rover.

Initially, there were three design concepts: a simple arm lift with a claw mechanism, a cart/basket with an arm and suction cup, and a cart/basket with a scooper. The third design concept was chosen due to its simplicity, which allowed for fewer moving parts, fewer degrees of freedom, and ease of development. We transitioned from a simple box chassis to a modular design with multiple layers, inspired by the rover kit provided at the beginning of the winter quarter. The modular design allowed for easy repairs and quick changes, such as lowering the height of the rover by adjusting the standoffs between the chassis layers. 3D printing was utilized to create components for the basket-lift and arm. All components of the subsystems were designed for 3D printing and effortless assembly. The basket-lift was modified to use a crank and slide mechanism, and the arm employed a rack and pinion design for translational motion, with an added degree of freedom for the scooper.

The rover was powered by a 12V battery, driving two 12V motors for the wheels and three 5V servo motors (two positional and one continuous). A buck converter was used to distribute power to the respective components, and precise control of the drive system was achieved using an L298N DC motor controller with an encoder. Control was divided as an Arduino Nano handled the low-level controls, such as moving the scooper or controlling the basket-lift, and a Raspberry Pi managed the high-computation tasks for navigation. The main component used for navigation was the Raspberry Pi and its camera, rather than ultrasonic or IR sensors.

Before assembling the rover, we calculated the move profile, drive system power requirements, and motor torque requirements to ensure our parts met the minimum power needs. FEA studies were performed on the chassis to ensure the rover could withstand a 6-inch drop, and motion studies ensured the arm and basket-lift would function properly for disk handling. A state diagram was also created to ensure all electronic components connected and functioned as intended, creating an understandable flow of power and facilitating easy wiring during assembly.

The autonomous performance relied heavily on the camera. When the camera operated smoothly, line tracking and object detection were nearly perfect. However, camera lag caused slow data transfer and image processing, leading to frequent positional corrections. The camera also required time to “warm up” after rebooting due to the Raspberry Pi’s memory being wiped, resulting in inconsistent trials. This led to varying degrees of satisfaction for our design requirements. Design requirements that did not involve the camera were met perfectly, while those involving the camera depended on its state of operation.

From this project, our team realized that our design relied too heavily on platform height and the performance of the Raspberry Pi camera. If we had more time, we would consider a design that did not depend on platform height, such as a small four-bar lift or an additional degree of freedom to elevate the basket-lift system for varying platform heights. Adding more sensors, such as IR and ultrasonic sensors, would provide backups in case the camera performed poorly and could divide

tasks among the sensors to improve overall reliability. Overall, our team's communication could have been improved, and we spent too much time deciding on a finalized design concept, leading to delays. While our rover was satisfactory, with more time, we could have perfected the autonomous section, making the rover perform flawlessly.

12. References

[1] Y. Chang. (2024). UCLA MAE-162D Week 4 Lab [Online]. Available: https://bruinlearn.ucla.edu/courses/178976/files/15976929?module_item_id=6534926

[2] Amazon. (2024). "Adeept RaspTank Smart Robot Kit." Amazon.com. https://www.amazon.com/Adeept-RaspTank-Raspberry-Educational-Programming/dp/B0B1X8D1F7/?_encoding=UTF8&pd_rd_w=2vTYs&content-id=amzn1.sym.35cab78c-35e3-4fc1-aab0-27eaa6c86063%3Aamzn1.symc.e5c80209-769f-4ade-a325-2eaec14b8e0e&pf_rd_p=35cab78c-35e3-4fc1-aab0-27eaa6c86063&pf_rd_r=5XB5WKM2BWFM7JK0J0M0&pd_rd_wg=mgVcR&pd_rd_r=ad6ec0ea-7562-41fb-89a6-93b4ca405e65&ref_=pd_gw_ci_mcx_mr_hp_atf_m (accessed Feb. 6, 2024).

[3] RobotShop. (2024). "Lynxmotion Aluminum A4WD1 Rover Kit." RobotShop.com. https://www.robotshop.com/products/4wd1-robot-aluminum-kit (accessed Feb. 6, 2024).

[4] drez20001. Iris Design #2 - A CNC Project. (July 30, 2016). Accessed: Feb. 1, 2024. [Online Video]. Available: https://www.youtube.com/watch?app=desktop&v=kJPcfT6eNb8

[5] P. Sanchez, A. Gonzalez-Garcia, and M. Sanchez, "VantTec Technical Design Report VTec U-III," VantTec, USA, 2020. Accessed: Feb. 27, 2024. Available: https://www.researchgate.net/figure/CAD-rendering-of-arm-and-gripper-system_fig3_343546455

[6] Elegoo, "Smart Robot Car Kit V4.0 (With Camera)." Elegoo.com. https://www.elegoo.com/products/elegoo-smart-robot-car-kit-v-4-0 (accessed Feb. 28, 2024).

[7] Hash Include Electronics. Line Follower Robot Using Arduino. (July 26, 2021). Accessed: Feb. 28, 2024. [Online Video]. Available: https://www.youtube.com/watch?app=desktop&v=5jh-5HGvC-I

[8] The Engineering Toolbox. "Friction – Friction Coefficients and Calculator." Engineeringtoolbox.com. https://www.engineeringtoolbox.com/friction-coefficients-d_778.html (accessed Feb. 28, 2024).

[9] University of Florida Mechanical Engineering Department. "Friction coefficients between different wheel/tire materials and concrete." mae.ufl.edu. https://mae.ufl.edu/designlab/Class%20Projects/Background%20Information/Friction%20coefficients.htm (accessed Feb. 28, 2024).

[10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics vol. 7, May 2022. https://docs.ros.org/en/humble/

13. Appendix

The following sections outline any information that was decided by the team to not provide vital information for the main body of the report but was still useful in the scope of the project. The figures, tables, equations, and variables of interest within the Appendix section is not included within the lists provided at the start of the document.

13.1. Initial Preliminary Calculations

This section is meant to highlight all the relevant calculations performed to determine various aspects of the robot's design. These include estimates for the device weight, power calculations, force calculations, and more.

13.1.1. Device Weight Estimate

The various individual components of the current robot design and their respective masses were gathered into a comprehensive table. The quantities for each component were specified in order to determine the total weight of each component, as well as the total weight of the robot. This estimated total weight is shown in the table below.

Table A1: Component Weights and Total Device Weight

Component	Weight (grams)	Quantity	Total Weight
Claw	85.61	1	85.61
Wheel	85.1	4	340.4
Raspberry PI	47	1	47
L298N Motor Driver	31.75	4	127
Sainsmart Nano	20	1	20
Front Camera	-	1	0
Mango Router	73.8	1	73.8
Motor (For Wheels)	90.2	2	180.4
Breadboard	38	1	38
LM2596	14	1	14
Battery	242.25	1	242.25
Chassis/Cage	1733.76	1	1733.76
	Total Weight		2902.22

With these general estimates for the weights of each component and the total weight of the robot, the center of gravity was generally located using SolidWorks. The SolidWorks model with the estimated center of gravity is shown below.

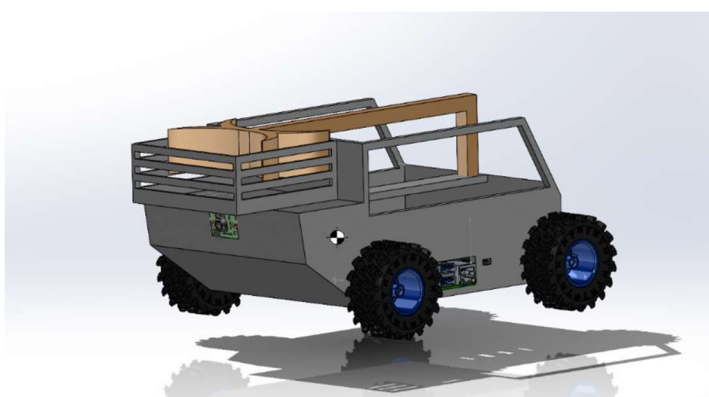

Figure A1: Current Robot Design CAD Model with Location for Center of Gravity

13.1.2. Drive System and Power Calculations

Wheel-to-Plywood Coefficient of Friction:

The rover's wheels are made of rubber material and the coefficient of friction between the rubber and the plywood of the track is approximately 0.705.

Comparison of Required Minimum Friction Coefficients for FWD, RWD, and AWD:

The required minimum coefficients of friction for the FWD, RWD, and AWD were calculated using the following equation for the respective wheel drive:

$$\mu = \frac{Force\ of\ Weight \times sin(Angle\ of\ Incline)}{Normal\ Force\ of\ Wheels}$$

The values for FWD, RWD, and AWD are 0.705, 0.432, and 0.268 respectively. Figure 11 compares the values of the friction coefficients as the horizontal distance of the center of mass from the rear increases from 0.1 m to 0.195 m (length of the wheelbase).

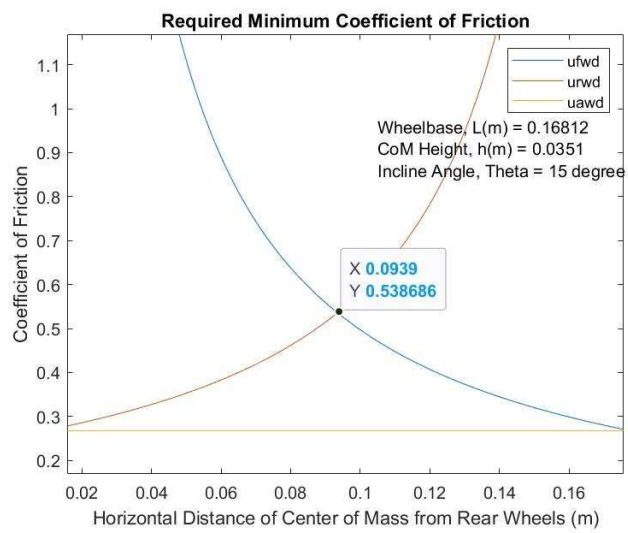

Figure A2: Required Minimum Coefficient of Friction

Normal Force between Rear Wheels and Ground:

The normal force between the rear wheels and the ground measured to be 17.038 and was calculated using the following equation:

$$N_r = \frac{(L - L_c)W_f cos(\theta) + h_c W_f sin(\theta)}{L}$$

where L represents the length of the wheelbase, Lc is the CoM distance from the rear wheels, hc is the CoM height from the ground, Wf is the weight force, and theta is the angle of incline.

Normal Force between Front Wheels and Ground:

The normal force acting on the front wheels of the rover measured to be 10.454. This force was calculated using the following equation:

$$N_f = \frac{L_c W_f \cos(\theta) - h_c W_f \sin(\theta)}{L}.$$

Total Tractive Force:

The total tractive force equated to 14.732 N by adding the FWD and RWD tractive force. The FWD and RWD tractive force were calculated by multiplying the normal force by the minimum friction coefficient of their respective wheel drive.

Rover Ability to Climb Slope:

To determine the capability of the transporter to climb the slope, the force of gravity along the slope is compared to the total tractive force. The force of gravity is 7.336 N and the total tractive force exceeds this value as it is double the gravitational force along the slope. The rover will have more than enough tractive force to overcome the resistance of gravity.

13.1.3. Move Profile and Motor Sizing Calculations

Maximum and Continuous Velocity:

A maximum and continuous velocity must be established for the segments of the track to perform motor sizing calculations and create a move profile. The range of the velocity the rover will move along the track was 5 to 15 cm/s. The speed of the robot traveling straight will be approximately 10 cm/s and the speed of the robot not traversing straight will be 8 cm/s.

Coefficient of Friction for the Wheels, and Rolling Coefficient of Friction:

The percent weight distribution between the rear and front wheels was determined using these two equations:

$$\beta_f = \frac{L_c}{L} + \frac{h_c \tan(\theta)}{L}$$
$$\beta_r = \frac{L - L_c}{L} + \frac{h_c \tan(\theta)}{L}$$

where βf and βr represent the percent weight distribution of the front and rear wheels respectively. βf was determined to be 0.38 and βr was 0.62. Since the robot will be utilizing front-wheel drive, the coefficient of friction used in the move profile and motor sizing calculations was 0.705, equivalent to the minimum required friction coefficient for FWD previously. The rolling coefficient utilized was 0.003.

Move Profile:

The track was divided into 5 segments, as shown in Figure 12 below, and a move profile was created with estimates for average and maximum velocities along each segment.

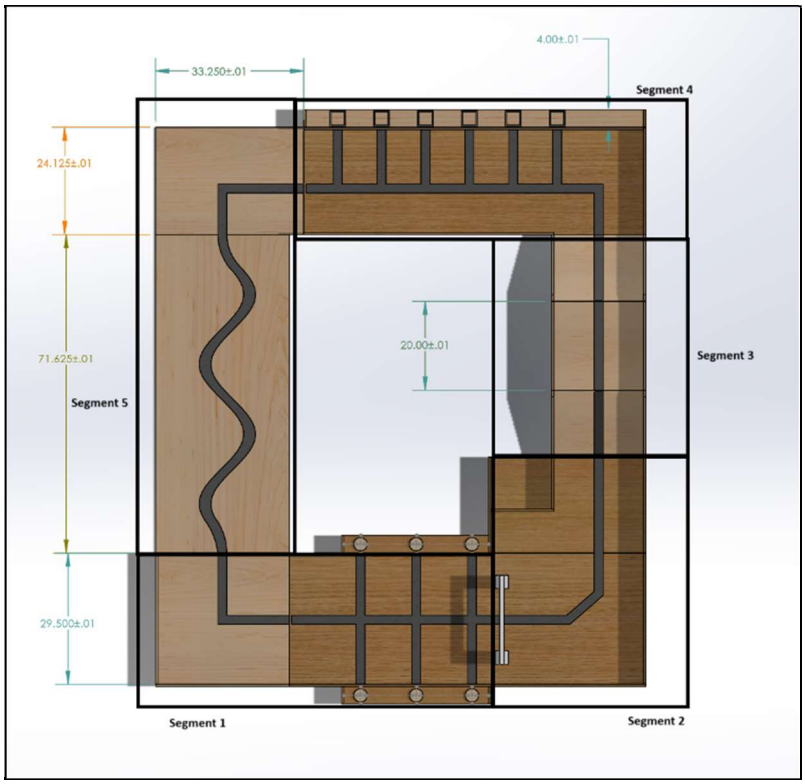

Figure A3: Track Divided into Segments

Table 6 presents the move profile type of each segment along with the acceleration time, constant velocity time, deceleration time, and total time. Table 7 displays the average maximum velocity and acceleration along each segment using the data from table 6 to calculate the values.

Table A2: The estimated amount of time for acceleration, constant velocity, and deceleration of each segment and each segment's move profile type

Path Segment	Move Profile Type	Accel Time (s)	Const Vel Time (s)	Decel Time (s)	Total Time (s)
1	Triangular	25	0	10.62	35.62
2	Trapezoidal	6	8.3	2.39	16.69
3	Trapezoidal	5.95	1	5.95	12.90
4	Triangular	22	0	11.76	33.76
5	Trapezoidal	5	22	8.74	35.74

Table A3: The estimated maximum velocity and acceleration for each segment

Path Segment	d_tot (m)	t_tot (S)	V_max (m/s)	a_max (m/s^2)
1	284.96	35.62	10	0.4
2	133.5	16.69	10	1.666666667
3	129.03	12.90	12	2.016806723
4	270.07	33.76	10	0.4545454545
5	285.94	35.74	10	1.428571429

With the data from tables 6 and 7, a move profile (shown in figure 13) was created with estimates of average and maximum velocities along each segment as the robot traverses the course.

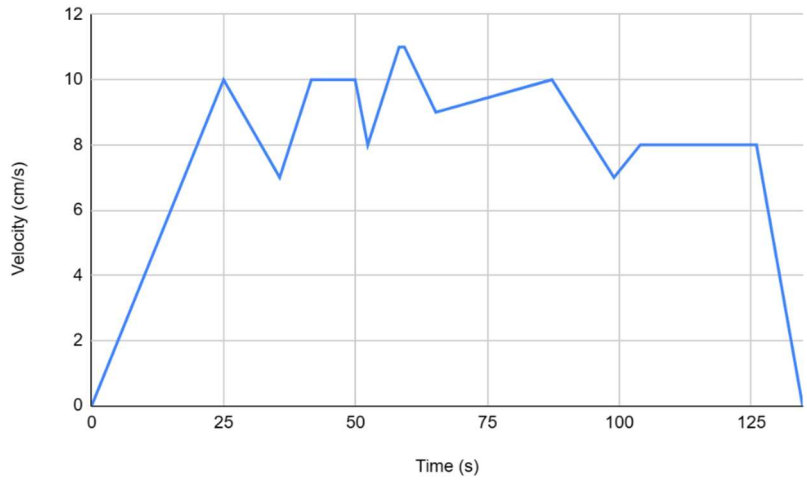

Figure A4: Move Profile

Propulsion force, Inertia force, Weight force, Friction force, and Rolling force along each segment:

Based on the move profile, the required propulsion force for each segment was calculated by summing the inertia force, weight force, friction force, and rolling friction force. Table # displays all the measured values for each respective force.

Table A4: The estimated maximum required propulsion force for each segment (summation of inertia force, weight force, wheel friction force, and rolling force)

Path Segment	F_i (N)	F_w (N)	F_f (N)	F_rol (N)	F_prop (N)
1	1.160888	28.441756	20.04176079	0.085325268	49.73
2	4.837033333	28.441756	20.04176079	0.085325268	53.41
3	5.853216807	28.441756	19.35885435	0.08241788	53.74
4	1.319190909	28.441756	20.04176079	0.085325268	49.89
5	4.146028571	28.441756	20.04176079	0.085325268	52.71

Power Required along each segment:

The required power along each segment was calculated by multiplying the required propulsion and the maximum velocity.

Table A5: The estimated required propulsion power for each segment

Path Segment	P_prop (W)
1	497.30
2	534.06
3	644.83
4	498.88
5	527.15

Propulsion torque required along each segment:

The required propulsion torque was calculated using the following equation:

$$T_{prop} = \frac{P_{prop} \times \frac{d_w}{2}}{\eta}$$

where Tprop is the required torque propulsion, Pprop is the required power, dw is the wheel diameter, and represents the system's efficiency. Table 10 displays the required torque propulsion values for each segment.

Table A6: Estimated maximum required propulsion power and motor torque along each segment

Path Segment	P_prop (W)	T_prop (N-m)
1	497.2973006	33.30
2	534.0587539	35.76
3	644.8349405	43.18
4	498.8803297	33.30
5	527.1487063	33.30

Drive Mechanism Type and Required Torque:

The drive mechanism for our robot is a front-wheel drive as turning is a priority and the required torque for front-wheel drive based on our calculations is 0.27113925 Nm. As long as the robot's motor torque is higher than 0.27113925 Nm, the robot should have no problem traversing the course, including the ramp section.